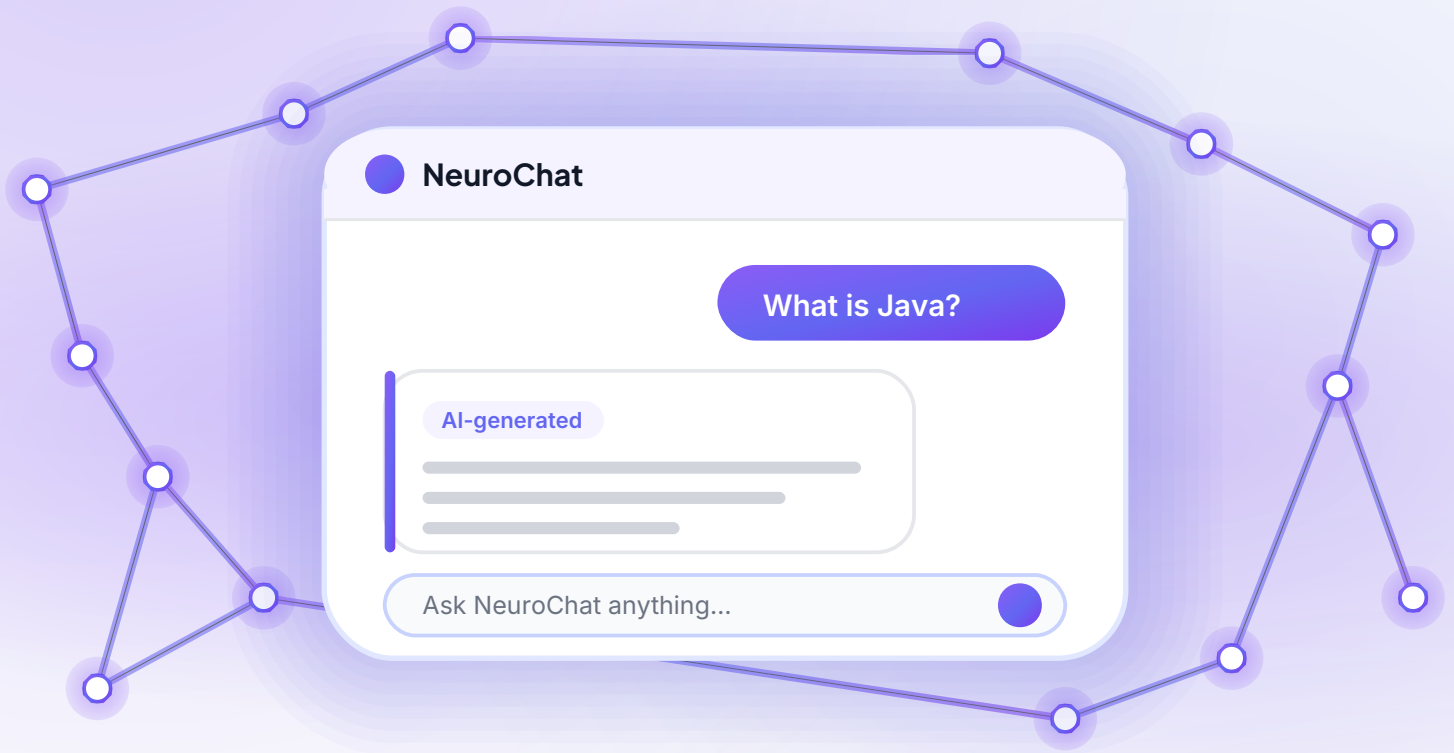




NeuroChat



ACADEMIC FIELD PROJECT · BCA SEMESTER 5

Master Project Specification

NeuroChat — design & development of a full-stack AI conversational assistant

“Your Intelligent Conversation Partner”

45

sections

79

requirements

15

build phases

17

API endpoints

73

test cases

13

diagrams specified

Contents

FOUNDATIONS

01	Executive Summary	5
02	Project Background.....	6
03	Previous Field Study Analysis	6
04	Problem Statement	7
05	Project Vision	7
06	Goals and Objectives	7
07	Target Users.....	8

REQUIREMENTS & FEATURES

08	Functional Requirements	9
09	Non-Functional Requirements.....	12
10	Previous Limitations → New Solutions	14
11	Feature Specification	15
12	User Journey.....	16
13	Complete Page Specification.....	16
14	UI/UX Design System.....	19

ARCHITECTURE & DESIGN

15	Authentication Architecture	23
16	Google OAuth Architecture	25
17	AI Architecture.....	28
18	Gemini Integration.....	29
19	Conversation Context Architecture.....	31
20	Multilingual Architecture.....	32
21	Voice Architecture	33
22	Database Design	34
23	API Specification	36
24	Frontend Architecture	39
25	Backend Architecture.....	41
26	Security Architecture	42
27	Error Handling	44
28	Accessibility	44
29	Responsive Design	45
30	Performance	46

QUALITY, DELIVERY & PLANNING

31	Testing Strategy	47
----	------------------------	----

32	Deployment Architecture	52
33	Project Folder Structure	54
34	Development Phases	56
35	MVP Scope	60
36	Future Scope	60
37	Diagrams Specification	61
ACADEMIC ALIGNMENT & CLOSING		
38	Academic Report Alignment	65
39	Viva Demonstration Plan	66
40	Master Requirements Traceability Matrix	68
41	Acceptance Criteria	70
42	Risks and Mitigations	71
43	Environment Configuration	72
44	Documentation Strategy	73
45	Final Implementation Checklist	74
APPENDICES		
A	Appendix: Internal Consistency Review	75
B	Appendix: Items You Must Fill In From Your Own Work	76

How to read this document

This is the **single source of truth** for building NeuroChat. Every later document in [docs/](#) (Section 44) is derived from it. If code and this document disagree, fix one of them deliberately, and record the change.

Status legend (used throughout so nothing is over-claimed, per the academic-honesty requirement):

Tag	Meaning
MVP	Required for submission. Must be built and demonstrated.
MVP+	Stretch. Build only after every MVP item works. Cheap, low-risk, browser-based.
Planned	Designed in this document (data model / extension points exist) but <i>not</i> built for submission.
Future	Idea only. Not designed in detail. Never described as existing.

No feature is currently "Implemented". When you finish a feature, update the status in [REQUIREMENTS.md](#), not here.

Priority scale (MoSCoW): **M**ust · **S**hould · **C**ould.

01 Executive Summary

NeuroChat is a full-stack, web-based AI conversational assistant. Users register (email/password or Google), hold multi-turn conversations with an AI, and find those conversations saved and searchable whenever they return.

It is the **practical implementation of a previous academic field study of ChatGPT** (Semester 3), which surveyed 80 participants and observed how people use conversational AI. That study *described* the technology and its limitations. NeuroChat *turns those findings into software requirements* and builds a working system that addresses them where technically feasible.

What NeuroChat is:

- A React single-page application with a light, softly glowing, premium visual identity.
- A Node.js/Express REST API that owns authentication, validation, data access, and all AI calls.
- A MongoDB database storing users, conversations and messages.
- An integration with the **Google Gemini API** (server-side only) for AI responses.

What NeuroChat is not:

- Not a ChatGPT clone. It has its own branding, UI and architecture.
- Not a new AI model. NeuroChat does **not** train or host a language model. The AI is provided by Gemini. The academic contribution is the **design and development of the complete application and its supporting architecture**.
- Not an always-correct system. It says so, in the interface.

Key decisions at a glance (each is justified in the section shown)

#	Decision	Chosen	Why (one line)	Section
D1	Frontend language	React + JavaScript (Vite)	Fewer moving parts for a first full-stack project; TypeScript noted as optional	24
D2	Session mechanism	JWT in an httpOnly cookie	Token unreachable by JavaScript, so XSS cannot steal it	15
D3	Cross-domain cookies	Frontend proxies <code>/api</code> to backend (same-origin)	Avoids third-party-cookie blocking in Safari/Chrome	32
D4	Google Sign-In	Google Identity Services ID-token flow , verified on server	Real, secure, simplest correct approach; no client secret needed	16
D5	Account linking	Link only on Google-verified email; drop unverified local password	Prevents account pre-hijacking	16
D6	Context handling	Sliding window with budget (last N messages within a size cap)	Simple, explainable, bounded cost; summarisation left as Planned	19
D7	Conversation title	First message, cleaned & truncated (no AI call)	Saves quota and latency	11, 22
D8	Message ownership	<code>userId</code> also stored on each message	One-query authorisation and search; costs 12 bytes per message	22
D9	Markdown	<code>react-markdown</code> + sanitiser, no raw HTML	Blocks XSS from AI output	26
D10	Real-time info	Not implemented ; retrieval hook defined	Honest; architecture ready	17

#	Decision	Chosen	Why (one line)	Section
D11	Email verification / forgot-password	Planned , not MVP	Needs an email service; scoped out honestly	15
D12	Styling	Plain CSS Modules + CSS variable tokens	Central theming, no build-time framework to explain	14, 24

02 Project Background

2.1 Origin

In Semester 3 the field work was a research/survey study titled around **ChatGPT and conversational AI**. It examined ChatGPT in education, business and productivity, user experience and satisfaction, and its advantages and limitations, using survey and observation methods with **80 participants**.

2.2 Semester 5 requirement

The Semester 5 field project must convert that topic into a **working application**. The chosen form is an AI conversational assistant with a real technology stack, real authentication and real persistence.

2.3 Fixed technology stack

Layer	Technology
Frontend	React (component-based, responsive)
Backend	Node.js + Express.js, REST API
Database	MongoDB (inspected with MongoDB Compass in the viva)
AI	Google Gemini API (server-side only)
Auth	Email/password + Google Sign-In (OAuth / OpenID Connect)

2.4 Scope boundary

The project is scoped for a **BCA Semester 5 submission**: ambitious enough to look like a real product, restrained enough that every feature can be finished, tested, and *explained* to an examiner.

03 Previous Field Study Analysis

The previous study's findings are treated as **requirements for improvement**, not as background reading.

#	Finding / recommendation from the previous study	Interpretation as a software problem	Where NeuroChat addresses it
F1	AI sometimes gives inaccurate or overly general answers	The system must never present AI output as authoritative, and must make verification easy	§17, §18 (AI-004, AI-005, AI-006, AI-011), landing "Trust" section
F2	AI may lack current/real-time information	The system must not pretend to have live data; it must be extensible to retrieval	§17.4 (AI-009, AI-010)
F3	Regional-language support should improve	Unicode-safe storage, language preference, Hindi/English UI, model-driven multilingual replies	§20 (FR-028, NFR-007)
F4	Voice interaction would improve accessibility	Optional, browser-based voice input and output, built as an isolated module	§21 (FR-029, FR-030)

#	Finding / recommendation from the previous study	Interpretation as a software problem	Where NeuroChat addresses it
F5	New users need guidance or tutorials	First-run onboarding with example prompts and a responsible-AI note	§13 (FR-027)
F6	Accessibility and usability should improve	WCAG-oriented design, keyboard support, responsive/mobile-first layout	§28, §29 (UI-004 to UI-008)
F7	Users value fast, conversational, context-aware responses	Multi-turn context, loading feedback, timeouts, efficient history loading	§19, §30 (FR-012, NFR-002)
F8	AI must be used responsibly; users should understand limits	Persistent disclaimer, sensitive-topic nudge, "AI-generated" labelling, honest marketing copy	§17.3, §13 (AI-004, AI-005)

Action for you (see Appendix B): the survey percentages and participant breakdown belong in your report. I have not invented any numbers. Where a statistic would strengthen a slide, this document marks [INSERT FROM SEMESTER 3 REPORT] .

04 Problem Statement

Conversational AI tools are widely used for study, work and everyday tasks, but the earlier field study found recurring problems: answers that can be inaccurate or too general, no awareness of current events, weak support for regional languages, no voice interaction, poor onboarding for first-time users, and uneven accessibility.

Most student projects in this area either (a) only *study* such tools, or (b) build a thin front-end over an AI API with no accounts, no saved history and no attention to the study's findings.

Problem: there is no small, secure, demonstrable application that (1) provides a genuine multi-turn AI conversation experience with per-user saved history, and (2) is deliberately *designed around* the documented limitations of conversational AI.

05 Project Vision

NeuroChat is a full-stack AI conversational assistant developed as the practical implementation of findings from a previous academic study of ChatGPT.

The product should feel like a modern AI SaaS product: bright, calm, softly glowing, fast to understand, honest about its limits. It must look and behave like something a user would choose to return to, not like a CRUD exercise.

Design personality: clean · bright · glowing · premium · trustworthy.

06 Goals and Objectives

6.1 Primary goal

Build and demonstrate a secure, working, multi-user AI chat application whose design demonstrably responds to the findings of the previous field study.

6.2 Objectives (measurable)

ID	Objective	Measure of success
01	Deliver a real authentication system (email/password + Google)	Registration, login, Google login and logout all work end to end in the demo
02	Deliver multi-turn AI conversation via Gemini, backend-only	A follow-up question using "its/that/it" is answered correctly; no API key appears in the browser bundle or network tab
03	Persist and manage conversation history per user	History survives refresh and re-login; search and delete work; another user's data is inaccessible
04	Address each of the eight previous-study findings	Requirement mapping in §10 and §38 is fully implemented or honestly marked Planned
05	Provide a light-first, glowing, accessible, responsive UI	Passes the UI test set in §31 on desktop and a 375 px-wide mobile viewport
06	Be secure to academic best-practice level	All SEC-* requirements verified by tests in §31.3
07	Be modular and extensible	Retrieval, voice and streaming can be added without rewriting existing modules
08	Be explainable in a viva	Every decision has a one-line rationale; a 5–7 minute demo script exists (§39)
09	Be documented	The docs/ set in §44 exists and matches the code
010	Be deployable	Live HTTPS deployment plus a local fallback (§32)

6.3 Non-goals

Training or fine-tuning a model · payments/subscriptions · admin dashboards · team/shared chats · native mobile apps · pixel-copying ChatGPT.

07 Target Users

User	Typical needs	How NeuroChat serves them
Students	Explanations, summaries, study help, Hindi/English support	Multi-turn explanation, multilingual prompts, saved study conversations, voice input
Developers	Code help, debugging, concept explanation	Code-block rendering with copy, follow-up context
Professionals	Drafting, summarising, brainstorming	Fast responses, organised history, search
Content creators	Ideas, scripts, rewriting	Copy response, retry for alternatives, multilingual output
General users	Everyday questions, curiosity	Onboarding, example prompts, simple interface
<i>Stakeholder: examiner/evaluator</i>	Understand what was built and why	Clear architecture, MongoDB Compass demo, requirement mapping

Assumed device mix: phone and laptop equally likely; hence a *designed* mobile experience (§29).

08 Functional Requirements

Master requirements table, part 1: Functional (FR) and AI (AI) requirements. Part 2 (NFR, SEC, UI) is in §9. Every requirement has an ID used later in tests (§31), phases (§34) and the traceability matrix (§40).

Columns: **Type** = Functional / AI. **Priority** = M/S/C. **Status tag** shown in the Requirement column where not **MVP**.

8.1 Functional requirements (FR)

ID	Requirement	Type	Pri	Description	Acceptance Criteria	Module
FR-001	Public landing page	Functional	M	Marketing page with hero, features, how-it-works, why-NeuroChat, use cases, trust/limitations, footer	All 7 sections render; CTAs route to register/login; no unverified claims of accuracy or live data	Landing
FR-002	User registration	Functional	M	Full name, email, password, confirm password; validation; strength meter; visibility toggle	Valid input creates a user and signs them in; every invalid input shows a specific message; loading state visible	Auth
FR-003	Duplicate email handling	Functional	M	Case-insensitive unique email	Second registration with same email (any letter case) is refused with a clear message; no second document in DB	Auth
FR-004	Email/password login	Functional	M	Email + password; generic failure message	Correct credentials open the app; wrong credentials show "Incorrect email or password" (never which one)	Auth
FR-005	Google Sign-In (new and existing users)	Functional	M	Real Google Identity Services flow verified by backend	New Google user gets a <code>users</code> document; returning user matches by <code>googleId</code> ; both land in the app	Auth / Google
FR-006	Safe account linking	Functional	M	Same email across local and Google handled without duplicates	Only Google-verified emails link; result is one user document; see §16.6	Auth / Google
FR-007	Logout	Functional	M	Ends session server-side (clears cookie) and client-side	After logout, protected API returns 401 and protected pages redirect to login	Auth
FR-008	Session persistence & current user	Functional	M	Session restored on refresh; optional "Remember me"	Refresh keeps user signed in; without "Remember me" session ends with browser; with it lasts 30 days	Auth
FR-009	Protected routes	Functional	M	Client route guards + server middleware	Unauthenticated visit to <code>/chat</code> redirects to <code>/login?redirect=/chat</code> ; after login returns there	Auth
FR-010	Create conversation	Functional	M	"New chat" creates an empty conversation lazily (on first message)	New chat shows welcome state; a DB document exists only after the first message is sent	Chat

ID	Requirement	Type	Pri	Description	Acceptance Criteria	Module
FR-011	Send message, receive AI response	Functional	M	Prompt goes to backend, then Gemini, response saved and shown	User and AI messages appear in order and persist; Enter sends	Chat / AI
FR-012	Multi-turn context	Functional	M	Follow-ups use earlier turns	"What is Java?" then "What are its advantages?" is answered about Java	Chat / AI
FR-013	Conversation persistence	Functional	M	All messages saved	After refresh and re-login, the full conversation is intact	Chat / DB
FR-014	List conversations	Functional	M	Sidebar lists user's conversations, newest activity first, paginated	Only the current user's conversations appear; ordering by <code>updatedAt</code> desc	Sidebar
FR-015	Open / continue conversation	Functional	M	Route <code>/chat/:id</code> loads history	Deep link works after refresh; another user's ID returns Not Found	Chat
FR-016	Automatic title	Functional	M	Title derived from first user message, no AI call	Title $\leq$ 60 chars, whitespace collapsed, Unicode-safe truncation	Chat
FR-017	Rename conversation	Functional	S	Inline edit	New title saved and shown; empty title rejected	Sidebar
FR-018	Search conversations	Functional	M	Debounced search over titles (<code>MVP</code>) and message content (<code>MVP+</code>)	Typing filters results within 400 ms of pause; results are only the user's	Sidebar
FR-019	Delete conversation with confirmation	Functional	M	Confirm dialog; deletes messages too	After confirm, conversation and its messages are gone from DB; cancel changes nothing	Sidebar
FR-020	Safe Markdown rendering	Functional	M	Paragraphs, headings, lists, code blocks, inline code, tables (GFM)	Sample Markdown renders correctly; <code><script></code> /HTML/ <code>javascript:</code> links are neutralised	Chat
FR-021	Copy response	Functional	M	Copy button on AI messages and code blocks	Clipboard contains the raw Markdown text; "Copied" feedback shown	Chat
FR-022	Retry response	Functional	M	Regenerate after failure or on demand	After a forced Gemini failure, Retry succeeds without duplicating the user message	Chat / AI
FR-023	Input behaviour	Functional	M	Multiline textarea; Enter sends; Shift+Enter newline; disabled while sending; max length	Behaviours verified; empty/whitespace-only blocked; > 4,000 chars blocked with counter	Chat
FR-024	Loading indicator and auto-scroll	Functional	M	"NeuroChat is thinking" indicator; scroll to newest message	Indicator shows within 100 ms; scroll does not fight a user who scrolled up	Chat
FR-025	Timestamps	Functional	S	Relative in sidebar, absolute on hover in chat	Locale-aware format shown	Chat
FR-026	Profile and settings	Functional	S	View name/email/avatar; change name; language; theme; logout	Changes persist and apply immediately	Settings
FR-027	Onboarding	Functional	M	First-run guide: welcome, how it works, example prompts, responsible-AI note; dismissible	Shows once for new user; dismissal stored in DB; can be re-opened from Settings	Onboarding

ID	Requirement	Type	Pri	Description	Acceptance Criteria	Module
FR-028	Language preference	Functional	S	UI language (English, Hindi MVP ; Marathi Planned); reply language: Auto / English / Hindi	Switching updates UI strings; "Auto" replies in the language of the prompt	i18n / AI
FR-029	Voice input	Functional	C (MVP+)	Browser speech recognition fills the input box	On supported browsers a mic button dictates text; unsupported browsers hide it gracefully	Voice
FR-030	Voice output	Functional	C (MVP+)	Browser speech synthesis reads a response	"Listen" button speaks the response and can stop it	Voice
FR-031	Theme control	Functional	C	Light default; optional dark	Choice persists; no flash of wrong theme on load	Theme
FR-032	Not Found / error pages	Functional	M	404 page and global error boundary	Unknown route shows friendly 404; render crash shows recovery screen, no stack trace	Errors
FR-033	Forgot / reset password	Functional	C (Planned)	Email-token reset	Not built for submission; token model documented in §15.9	Auth
FR-034	Email verification	Functional	C (Planned)	Verify email ownership at registration	Not built for submission; consequences handled by §16.6	Auth

8.2 AI requirements (AI)

ID	Requirement	Type	Pri	Description	Acceptance Criteria	Module
AI-001	Backend-only Gemini access	AI	M	All Gemini calls originate from the server	Built frontend bundle and network tab contain no key; grep of dist/ finds none	AI service
AI-002	Bounded context strategy	AI	M	Sliding window within message-count and size budget	A 200-message conversation still produces a request under budget; newest messages always included	AI service
AI-003	System instruction	AI	M	Assistant identity, honesty, uncertainty, language-matching, formatting	Instruction stored in one config file; responses admit uncertainty rather than invent	AI service
AI-004	"AI-generated" indication and disclaimer	AI	M	Badge on each AI message; persistent line under input	Visible on every AI message and on mobile	Chat UI
AI-005	Sensitive-topic verification nudge	AI	S	Heuristic detects medical/legal/financial/safety topics and shows a verify-with-a-professional note	Prompts like "what dose of ..." trigger the note; ordinary prompts do not. Documented as a <i>heuristic</i> , not a guarantee	Chat / AI
AI-006	Friendly error mapping with retry	AI	M	Gemini errors mapped to user messages (§27)	Simulated 429, 503, timeout each show a distinct friendly message with Retry	AI service
AI-007	Timeout and malformed-response handling	AI	M	30 s request timeout; empty/invalid response rejected	Forced empty response yields error state, not a blank bubble	AI service

ID	Requirement	Type	Pri	Description	Acceptance Criteria	Module
AI-008	Provider abstraction	AI	S	<code>aiProvider</code> interface; Gemini is one implementation	Swapping model name needs only env change; provider swap touches one file	AI service
AI-009	Retrieval extension point	AI	C (<code>Planned</code>)	Empty <code>retrieval</code> module interface and message <code>sources</code> field	Interface and field exist; no UI or docs claim web access	AI service
AI-010	No false capability claims	AI	M	Marketing and UI never claim live data, guaranteed accuracy, or an own model	Copy review checklist passed (\$45)	All
AI-011	Safety-block handling	AI	M	Handle Gemini safety/finish reasons and empty candidates	Blocked response shows a neutral message; the user's message stays saved and Retry/rephrase is possible	AI service

09

Non-Functional Requirements

Master requirements table, part 2.

9.1 Non-functional (NFR)

ID	Requirement	Type	Pri	Description	Acceptance Criteria	Module
NFR-001	Page performance	NFR	S	Fast first load	Landing page Lighthouse Performance ≥ 85 on a laptop/broadband profile; route-level code splitting used	Frontend
NFR-002	API responsiveness	NFR	M	Non-AI endpoints quick; AI calls bounded	Non-AI endpoints p95 < 500 ms on deployed stack (warm); AI call timeout 30 s	Backend
NFR-003	Scalable history	NFR	M	No unbounded reads	Lists and messages are paginated; indexes exist for all query paths	DB
NFR-004	Maintainability	NFR	M	Layered structure; single-responsibility modules	Routes contain no business logic; no file > ~300 lines without reason	All
NFR-005	Graceful degradation	NFR	M	Partial failure does not break the app	With Gemini down, users can still log in, browse and search history	All
NFR-006	Browser support	NFR	M	Latest Chrome, Edge, Firefox, Safari	Manual smoke test on each; voice features degrade gracefully	Frontend
NFR-007	Unicode and multilingual safety	NFR	M	UTF-8 end to end	Hindi/Marathi text round-trips through UI, API, DB, Compass without corruption	All
NFR-008	Documentation	NFR	M	Doc set kept current	All files in \$44 exist before submission	Docs
NFR-009	Testability	NFR	S	App factory separable from server start; Gemini mockable	Automated tests run without real Gemini calls	Backend

ID	Requirement	Type	Pri	Description	Acceptance Criteria	Module
NFR-010	Privacy transparency	NFR	M	Users told what is stored and that messages go to Google's Gemini API	Privacy page states data flows accurately (§26.9)	Legal pages

9.2 Security requirements (SEC)

ID	Requirement	Type	Pri	Description	Acceptance Criteria	Module
SEC-001	Password hashing	Security	M	bcrypt (cost ≥ 12), never store plaintext	DB shows only hashes; hash differs for identical passwords (salt)	Auth
SEC-002	Secure session token	Security	M	JWT in httpOnly, Secure, SameSite cookie	Token not readable via <code>document.cookie</code> ; not in localStorage	Auth
SEC-003	Authentication middleware	Security	M	Every non-public route verifies token	No token, invalid or expired token returns 401	Middleware
SEC-004	Ownership authorisation	Security	M	Every conversation/message query scoped by <code>userId</code>	User B requesting user A's conversation ID gets 404; verified by automated test	Controllers
SEC-005	Server-side validation	Security	M	Schema validation on every input	Malformed/oversized/wrong-type body returns 400 with field messages	Validators
SEC-006	Secret management	Security	M	Secrets only in environment variables; <code>.env</code> git-ignored	Repo history contains no secret; <code>.env.example</code> has placeholders only	Config
SEC-007	CORS allow-list	Security	M	Only the known client origin, credentials enabled	Request from another origin is blocked	Server
SEC-008	Rate limiting	Security	M	Stricter on auth, per-user on messages	11th login attempt in window returns 429	Middleware
SEC-009	XSS-safe rendering	Security	M	No raw HTML from AI output	Injection test strings render as text	Chat UI
SEC-010	Server-side Google token verification	Security	M	Verify signature, audience, issuer, expiry, <code>email_verified</code>	Tampered or wrong-audience token is rejected	Google auth
SEC-011	Security headers	Security	S	<code>helmet</code> defaults	Response headers include standard hardening headers	Server
SEC-012	Non-leaking auth errors and CSRF defence	Security	M	Generic messages; SameSite plus Origin/custom-header check on state-changing requests	Login error identical for unknown email and wrong password; cross-site POST rejected	Auth
SEC-013	NoSQL-injection resistance	Security	M	Validate types; never pass raw body objects into queries	<code>{"email":{"\$gt":""}}</code> returns 400	Validators
SEC-014	No secrets in frontend	Security	M	Only public config (<code>VITE_*</code>) in client	Build output scanned; only Google <i>client ID</i> and API base path present	Build

9.3 UI / UX requirements (UI)

ID	Requirement	Type	Pri	Description	Acceptance Criteria	Module
UI-001	Light-first default	UI	M	Default theme is light regardless of OS setting	First load is light on a dark-mode OS	Theme
UI-002	Central design tokens	UI	M	All colours, radii, shadows, glow via CSS variables	Changing <code>--color-primary-*</code> in one file re-themes the app; no hard-coded hex in components	Styles
UI-003	Subtle premium glow	UI	M	Glow used on focus, primary CTA, hero, active states only	Design review checklist §14.10 passed; text never sits on the glow halo	Styles
UI-004	Responsive layouts	UI	M	Four breakpoints, designed (not shrunk)	Verified at 360, 768, 1024, 1440 px	Layouts
UI-005	Accessibility (WCAG 2.1 AA target)	UI	M	Semantic HTML, keyboard, focus, contrast	axe scan zero critical issues; keyboard-only run-through of the demo succeeds	All
UI-006	Reduced motion	UI	S	Respect <code>prefers-reduced-motion</code>	Animations disabled or minimal when set	Styles
UI-007	Complete state coverage	UI	M	Loading, empty, error, success on every data view	Checklist per page in §13 satisfied	All
UI-008	Mobile navigation	UI	M	Sidebar becomes an off-canvas drawer	Drawer opens/closes by button, overlay, and Esc; focus trapped while open	Layout
UI-009	Branding assets	UI	S	Logo, favicon, wordmark, social preview	Assets exist in SVG and PNG sizes	Assets
UI-010	No theme flash	UI	S	Theme applied before first paint	No visible flash when reloading in dark theme	Theme

10

Previous Limitations → New Solutions

Limitation from previous study	Design response in NeuroChat	Requirement IDs	Status
Inaccurate/general answers	System instruction that prefers stating uncertainty; "AI-generated" badge; disclaimer; verification nudge for sensitive topics; retry; architecture for citations	AI-003, AI-004, AI-005, FR-022, AI-009	MVP except citations Planned
No current information	Explicitly <i>not claimed</i> . Retrieval module interface reserved; Phase 2 design in §17.4	AI-009, AI-010	Planned
Regional-language support	Unicode storage, Hindi/English UI, language preference, model replies in user's language	FR-028, NFR-007	MVP (Marathi Planned)
No voice	Web Speech API input/output as isolated module	FR-029, FR-030	MVP+
Need for onboarding	First-run guide with examples and responsible-AI note	FR-027	MVP

Limitation from previous study	Design response in NeuroChat	Requirement IDs	Status
Accessibility/usability	Accessibility spec, responsive design, keyboard support	UI-004 to UI-008	MVP
Value of fast, contextual answers	Context window strategy, loading feedback, timeout, pagination	FR-012, AI-002, NFR-002	MVP
Responsible use	Trust section on landing, disclaimer, honest copy checklist	AI-004, AI-010	MVP

Requirement IDs are defined only in §8–§9.

11 Feature Specification

Features are grouped into modules. Each lists behaviour, rules and edge cases.

11.1 Landing & marketing

Sections and copy rules are in §13.1. **Rule:** no sentence may claim guaranteed accuracy, live information, or a proprietary AI model.

11.2 Authentication

Registration, login, Google Sign-In, logout, session restore. Details in §15 and §16. **Password policy:** 8–72 characters (72 is bcrypt's byte limit), at least one letter and one number; strength meter is guidance only, server enforces the policy.

11.3 Chat

- **Lazy conversation creation:** "New chat" resets the UI to the welcome state. The database document is created when the first message is sent, so empty conversations never pile up.
- **Send flow:** input trimmed; blocked if empty or over 4,000 characters; UI optimistically shows the user message; on success the AI message is appended; on failure the user message stays and an inline error with **Retry** appears.
- **Retry:** re-requests an AI response for the last user message. It never duplicates the user message.
- **Copy:** copies raw Markdown, not rendered HTML.
- **Verification nudge (AI-005):** when the heuristic flags a sensitive topic, a subtle banner appears *under that response*: "This looks like a health/legal/financial/safety topic. Please verify with a qualified professional or an official source."

11.4 Conversation management

- **Title:** first user message → strip Markdown and newlines → collapse whitespace → truncate to 60 *characters* (using `Array.from` so Devanagari and emoji are not cut mid-character) → append "..." if truncated.
- **Rename:** inline edit, 1–100 characters.
- **Search:** debounced 300 ms. MVP searches titles with an escaped, case-insensitive match. **MVP+** adds message-content search using a MongoDB text index.
- **Delete:** confirmation dialog naming the conversation; deletes messages then the conversation; irreversible and stated as such.
- **Pagination:** sidebar loads 20 at a time with "Load more"; messages load latest 50 and page backwards on scroll up.

11.5 Profile & settings

Name, email (read-only), avatar (Google picture or initials), UI language, reply language, theme, "Show onboarding again", logout.

11.6 Onboarding

Three dismissible cards on first login: (1) Welcome and what NeuroChat can do; (2) How conversations work — context, history, titles; (3) Responsible use — verify important answers, AI can be wrong — plus four clickable example prompts. Completion stored as `settings.onboardingCompleted`.

11.7 Multilingual and voice

See §20 and §21.

12 User Journey

12.1 Primary journey

Landing Page

- Register / Login / Continue with Google
- Authenticated Home (chat dashboard, empty state + onboarding on first run)
- New Conversation
- Send Prompt
- AI Response (labelled AI-generated)
- Continue Conversation (context preserved)
- Conversation automatically saved
- View previous conversations
- Search / rename / delete
- Logout

12.2 Journey with states

Step	Success path	Failure paths handled
Register	Account created, auto sign-in, onboarding	Duplicate email, weak password, network error, rate limit
Login	Redirect to <code>?redirect</code> target or <code>/chat</code>	Wrong credentials, rate limit, Google popup closed/blocked
Send prompt	Message shown at once, thinking indicator, AI reply	Gemini down/limited/timeout → inline error + Retry; session expired → re-login and return
Return later	History visible, last chat openable	Deleted/foreign conversation ID → Not Found page
Logout	Landing page; cookie cleared	Logout with expired session still ends cleanly

12.3 First-time-user journey (addresses F5)

Register → onboarding cards → click an example prompt → first AI response → tooltip pointing to the sidebar "Your conversations are saved here."

13 Complete Page Specification

Route map

Route	Page	Access
/	Landing	Public (signed-in visitors see "Open NeuroChat" instead of "Start Chatting")
/login	Login	Public (signed-in users redirect to <code>/chat</code>)
/register	Registration	Public (same rule)
/chat	Chat dashboard (new/empty conversation)	Protected
/chat/:conversationId	Conversation view	Protected
/settings	Profile & Settings	Protected
/privacy, /terms	Static legal pages	Public
*	Not Found	Public
/forgot-password	Planned	Public

Google authentication is **not a separate page**. It is a component embedded in Login and Register (§16).

13.1 Landing Page (/)

- **Purpose:** explain and sell NeuroChat honestly; drive registration.
- **Layout (top to bottom):** sticky glass navbar → Hero → Features → How It Works → Why NeuroChat → Use Cases → Trust & Limitations → final CTA band → Footer.
- **Components:** `Navbar`, `Hero`, `HeroVisual`, `FeatureCard` ×7, `StepCard` ×4, `WhyCard` ×4, `UseCaseTabs` /cards×5, `TrustPanel`, `CtaBand`, `Footer`.
- **Hero:** headline (e.g., "Think it. Ask it. Continue the conversation."), sub-copy, primary CTA **Start Chatting** → `/register`, secondary **Learn More** → smooth-scroll to Features, plus Login/Register in navbar. `HeroVisual` is a glowing, layered SVG/CSS scene: a light glass chat card with an animated typing indicator over a soft indigo-violet radial glow. Decorative only (`aria-hidden`).
- **Features (7):** Intelligent conversations · Context-aware responses · Conversation history · Secure authentication · Multilingual interaction · Responsive experience · AI-powered assistance. Copy states capabilities only; for voice, use "Voice input and read-aloud (on supported browsers)" only if **MVP+** is built.
- **How it works (4):** Create your account → Ask NeuroChat anything → Continue the conversation → Your conversations are saved securely.
- **Why NeuroChat (ties to study):** four cards derived from findings F1, F3/F4, F5, F6/F7, phrased as design goals, not as achievements. Example: "Built from research: our earlier study of 80 users showed people want clearer guidance and honest limits, so we built both in." `[INSERT ONE REAL STATISTIC FROM SEMESTER 3 REPORT IF DESIRED]`.
- **Use cases:** Students, Developers, Professionals, Content creators, General users, each with one example prompt.
- **Trust & Limitations panel (mandatory):** "AI can be wrong. Answers may be inaccurate, incomplete or out of date. NeuroChat does not browse the web. For medical, legal, financial or safety decisions, verify with a qualified professional or official source."
- **Footer:** logo, tagline, links (Features, How it works, Login, Register), Privacy, Terms, About (project info: BCA Semester 5 field project), Contact (college email placeholder).
- **API calls:** none (optionally `GET /api/auth/me` to switch CTAs when signed in).
- **Loading / error:** static; fonts use `font-display: swap`; hero visual is CSS so it never fails to load.
- **Responsive:** hero stacks (text above visual) below 900 px; feature grid 3→2→1 columns; navbar collapses to a menu button below 768 px.

13.2 Login (/login)

- **Purpose:** authenticate returning users.
- **Layout:** centred glass card on a soft glowing background (desktop: optional split with brand panel; mobile: single column).
- **Components:** `AuthCard`, `TextField`, `PasswordField` (visibility toggle), `Checkbox` (Remember me), `PrimaryButton`, `GoogleSignInButton`, `Divider("or")`, `FormError` (live region), link to Register. "Forgot password?" link hidden until **Planned** feature exists (never link to a dead page).
- **User actions:** submit credentials; toggle password visibility; Google Sign-In; go to Register.
- **API:** `POST /api/auth/login`, `POST /api/auth/google`.
- **Loading:** button shows spinner and is disabled; Google button disabled during verification.
- **Errors:** field-level (email format, required); form-level ("Incorrect email or password", "Too many attempts, try again in N minutes", "Network problem"); Google-specific ("Google sign-in was cancelled", "Couldn't verify your Google account").
- **Redirect:** to `redirect` query param (validated: must start with `/` and not `//`) else `/chat`.
- **Responsive:** card max-width 440 px; inputs ≥ 44 px tall.

13.3 Registration (/register)

- **Purpose:** create an account.
- **Fields:** Full name (2–60), Email, Password, Confirm password.
- **Components:** as Login plus `PasswordStrengthMeter` (text + bar, not colour alone), `ValidationHints` list ("8+ characters", "a letter", "a number").
- **API:** `POST /api/auth/register`, `POST /api/auth/google`.
- **Loading / error:** as Login; duplicate email shows "An account with this email already exists. Log in instead?" with link; if that account is Google-based: "This email uses Google sign-in. Continue with Google."
- **Success:** auto sign-in, brief success toast, redirect to `/chat` with onboarding.

13.4 Google authentication flow

- **Purpose:** real Google login, embedded in Login/Register.
- **Components:** `GoogleSignInButton` (Google Identity Services), `useGoogleAuth` hook.
- **Actions:** click → Google account chooser → credential returned to page → sent to backend.
- **API:** `POST /api/auth/google`.
- **States:** loading, popup blocked/closed, backend rejection, success. Full specification in §16.

13.5 Chat Dashboard (/chat) and 13.6 Conversation View (/chat/:id)

One layout (`AppLayout`) hosting both.

- **Purpose:** the product itself.
- **Layout (desktop ≥ 1024 px):** left **Sidebar** (280 px) + main column: top bar (conversation title, mobile menu button) → scrollable **MessageList** → sticky **Composer** with disclaimer line.
- **Sidebar components:** `Logo`, `NewChatButton`, `SearchBox` (debounced), `ConversationList` (`ConversationItem` with rename/delete menu), "Load more", `UserMenu` (avatar, name, Settings, theme, Logout).
- **Main components:** `WelcomeState` (greeting, 4 example prompts, onboarding entry), `MessageList`, `MessageBubble` (user/AI variants), `MarkdownRenderer`, `CodeBlock` (copy), `MessageActions` (Copy, Retry, **MVP+** Listen), `AiBadge`, `VerificationNote`, `TypingIndicator`, `ErrorBanner` (with Retry), `Composer` (textarea, char counter near limit, send button, **MVP+** mic), `ConfirmDialog`, `OnboardingModal`.

- **Actions:** new chat; send; Enter/Shift+Enter; copy; retry; select/rename/delete/search conversation; load older messages; logout.
- **API:** `POST /api/conversations` (at first send), `GET /api/conversations`, `GET /api/conversations/:id`, `GET /api/conversations/:id/messages`, `POST /api/conversations/:id/messages`, `POST .../messages/retry`, `PATCH / DELETE /api/conversations/:id`.
- **Loading:** skeleton items in sidebar; skeleton bubbles when opening a conversation; typing indicator during AI wait.
- **Empty:** no conversations → "Your conversations will appear here"; no search results → "No conversations match".
- **Errors:** conversation not found → Not Found view inside layout; AI error → inline banner with Retry; history load error → "Couldn't load your conversations" with Retry; 401 → re-login flow.
- **Responsive:** < 1024 px the sidebar is an off-canvas drawer (button in top bar, overlay, Esc closes, focus trapped); message max-width 100% minus 16 px padding on mobile, 760 px readable column on desktop; composer respects the on-screen keyboard (`100dvh` , safe-area insets).

13.7 Profile / Settings (`/settings`)

- **Purpose:** account info and preferences.
- **Components:** `ProfileCard` (avatar, name edit, email read-only, sign-in method), `PreferenceGroup` (UI language, reply language, theme), `OnboardingReset`, `AboutAi` (short limitations text), `LogoutButton`.
- **API:** `GET /api/users/me`, `PATCH /api/users/me`.
- **States:** saving indicator, success toast, validation errors, load error with retry.
- **Responsive:** single column, sections stack.

13.8 Privacy & Terms (`/privacy` , `/terms`)

Plain static content, accurate to §26.9. Required because the footer links to them and because user text is sent to a third-party AI provider.

13.9 Not Found / Error (`*` , `ErrorBoundary`)

- **Purpose:** graceful dead ends.
- **Components:** glowing "404" illustration (decorative), explanation, buttons "Go home" and "Open chat". Error boundary variant: "Something went wrong" + reload; never shows a stack trace.

13.10 Page state checklist (UI-007)

Every page above must be verified for: loading · empty · error · success · keyboard · mobile.

14 UI/UX Design System

14.1 Design principles

1. **Light-first.** Default theme is light. Dark is optional and secondary (§14.11).
2. **Glow is seasoning, not the meal.** Glow appears on hero visuals, primary CTA, focus, active/selected states, and the AI avatar. Never behind body text.
3. **Text is always readable.** Gradients are decorative surfaces or large elements. Body text sits on white/near-white surfaces.
4. **Calm density.** Generous spacing, clear hierarchy, few simultaneous colours.

5. Motion has a purpose. Feedback and continuity only; reduced-motion respected.

14.2 Brand identity

Item	Specification
Name	NeuroChat
Tagline	Your Intelligent Conversation Partner
Logo concept	A rounded speech bubble whose interior holds three small connected nodes (a "neuron" cluster) forming a soft triangle; the bubble outline is the indigo→violet gradient, nodes are white on a gradient fill or gradient on white. Signals <i>conversation</i> + <i>intelligence</i> without copying any existing AI brand. Provide SVG in horizontal (mark + wordmark), stacked and mark-only versions.
Wordmark	"Neuro" in the primary gradient + "Chat" in gray-900, Plus Jakarta Sans 700
Favicon	Mark-only simplified to a gradient bubble with two nodes on a white rounded square; provide 32×32, 180×180 (Apple touch), SVG
Social preview	1200×630, hero glow with wordmark and tagline
Voice/tone	Friendly, plain, honest. No hype words such as "revolutionary" or "always accurate".

14.3 Colour palette

Contrast is against the stated background. Success green is for icons; verify all pairs.

Role	Token	Value	Notes
Indigo (primary)	---color-indigo-500	#6366F1	Buttons and links , white text 4.47:1
Indigo dark (hover)	---color-indigo-600	#4F46E5	White text on it ≈ 6.3:1
Violet (secondary)	---color-violet-500	#8B5CF6	White text ≈ 4.2:1
Lavender (accent)	---color-lavender-100	#EDE9FE	Badges, selected states
Background	--bg-page	#F8FAFC	Off white
Background soft	--bg-soft	#EDE9FE	Section bands
Surface / card	--surface	FFFFFF	
Text primary	--text-primary	#111827	≈ 17:1 on white
Text secondary	--text-secondary	#6B7280	≈ 4.8:1 on white
Text muted	--text-muted	#6B7280	≈ 4.8:1; not for essential small text
Border	--border	#E5E7EB	
Border strong	--border-strong	#D1D5DB	Inputs
Success	--success	#16A34A	
Warning	--warning	#B45309	
Error	--error	#DC2626	
Info	--info	#4F46E5	

When you implement, **re-check every text/background pair** with a contrast checker (e.g., WebAIM). The values above are close to AA (4.5:1) and must be verified in the final build.

14.4 Gradients

Token	Value	Use
<code>--gradient-primary</code>	<code>linear-gradient(135deg, #6366F1 0%, #8B5CF6 100%)</code>	Primary button, user bubble, active states (white text, 600 weight)
<code>--gradient-brand-soft</code>	<code>linear-gradient(135deg, #F5F3FF 0%, #EDE9FE 100%)</code>	Section backgrounds, cards' subtle wash
<code>--gradient-text</code>	<code>linear-gradient(135deg, #6366F1, #8B5CF6)</code>	Headline highlight word, wordmark (large text only)
<code>--gradient-glow</code>	<code>radial-gradient(60% 60% at 50% 40%, rgba(139,92,246,.24), rgba(99,102,241,.14) 50%, transparent 100%)</code>	Hero halo, decorative blobs

14.5 Design tokens (authoritative CSS variable set)

```
:root {
  /* colour */
  --color-indigo-500:#6366F1; --color-indigo-600:#4F46E5;
  --color-violet-500:#8B5CF6; --color-lavender-100:#EDE9FE;
  --bg-page:#F8FAFC; --bg-soft:#EDE9FE; --surface:#FFFFFF; --surface-glass:rgba(255,255,255,.72);
  --text-primary:#111827; --text-secondary:#6B7280; --text-muted:#6B7280; --text-on-primary:#FFFFFF;
  --border:#E5E7EB; --border-strong:#D1D5DB;
  --success:#16A34A; --warning:#B45309; --error:#DC2626; --info:#4F46E5;

  /* message surfaces */
  --msg-ai-bg:#FFFFFF; --msg-ai-border:#E5E7EB; --msg-ai-accent:var(--color-indigo-500);
  --msg-user-bg:var(--gradient-primary); --msg-user-text:#FFFFFF;

  /* radius */
  --radius-sm:8px; --radius-md:12px; --radius-lg:16px; --radius-xl:24px; --radius-pill:999px;

  /* shadow (soft, indigo-tinted) */
  --shadow-sm:0 1px 2px rgba(17,24,39,.06);
  --shadow-md:0 4px 16px rgba(99,102,241,.08);
  --shadow-lg:0 12px 40px rgba(99,102,241,.12);

  /* glow (subtle) */
  --glow-intensity:.16;
  --glow-sm:0 0 1px rgba(99,102,241,.10), 0 4px 18px rgba(139,92,246,var(--glow-intensity));
  --glow-md:0 0 1px rgba(99,102,241,.14), 0 8px 30px rgba(99,102,241,.20);
  --focus-ring:0 0 3px #fff, 0 0 5px #6366F1;

  /* spacing scale (4px base) */
  --space-1:4px; --space-2:8px; --space-3:12px; --space-4:16px; --space-6:24px; --space-8:32px; --space-12:48px; --space-16:64px;

  /* type */
  --font-heading:'Plus Jakarta Sans','Noto Sans Devanagari',system-ui,sans-serif;
  --font-body:'Inter','Noto Sans Devanagari',system-ui,sans-serif;
  --font-mono:'JetBrains Mono',ui-monospace,Menlo,Consolas,monospace;

  /* motion */
  --ease:cubic-bezier(.2,.8,.2,1); --dur-fast:120ms; --dur:200ms; --dur-slow:360ms;

  /* layout */
  --sidebar-width:280px; --content-max:760px; --touch-min:44px;
}
@media (prefers-reduced-motion: reduce){ :root{ --dur-fast:0ms; --dur:0ms; --dur-slow:0ms; } }
```

14.6 Typography

Style	Font	Size / line-height	Weight
Display (hero)	Heading	clamp(2.25rem, 5vw, 3.75rem) / 1.1	800
H1	Heading	2rem / 1.2	700
H2	Heading	1.5rem / 1.3	700
H3	Heading	1.25rem / 1.4	600
Body	Body	1rem (16px) / 1.6	400
Small	Body	0.875rem / 1.5	400
Chat message	Body	1rem / 1.65	400
Code	Mono	0.875rem / 1.6	400

Devanagari fallback ([Noto Sans Devanagari](#)) is loaded so Hindi/Marathi text looks intentional, not like a broken fallback. Minimum body text is 16 px. Never disable zoom.

14.7 Component styles

Component	Specification
Primary button	<code>--gradient-primary</code> background, white text 600, height 44 px, <code>--radius-md</code> , <code>--glow-sm</code> at rest, <code>--glow-md</code> on hover with 1 px lift, focus ring token, disabled = 50% opacity + <code>not-allowed</code> + no glow
Secondary button	White surface, 1 px <code>--border-strong</code> , text <code>---indigo</code> , hover soft lavender wash
Ghost/icon button	Transparent, 44×44 touch target, hover <code>--bg-soft</code> ; must have <code>aria-label</code>
Card	<code>--surface</code> , 1 px <code>--border</code> , <code>--radius-lg</code> , <code>--shadow-md</code> ; hover raises to <code>--shadow-lg</code> only when interactive
Glass card (auth, navbar)	<code>--surface-glass</code> + <code>backdrop-filter: blur(14px)</code> ; provide a solid <code>--surface</code> fallback where <code>backdrop-filter</code> is unsupported
Input	White, 1 px <code>--border-strong</code> , <code>--radius-md</code> , 44 px min height, label above (never placeholder-only); focus = indigo edge + <code>--glow-sm</code>
Chat composer	Rounded container (<code>--radius-xl</code>), <code>--shadow-md</code> ; on focus-within gets <code>--glow-md</code> ; send button is a gradient circle
User message bubble	<code>--msg-user-bg</code> gradient, white text, <code>--radius-lg</code> with reduced bottom-right corner, right-aligned, max-width 80% (mobile 90%)
AI message	White card, 1 px border, 3 px left accent in indigo→violet, NeuroChat avatar with soft glow, <code>AiBadge</code> "AI-generated", actions row below
Code block	<code>--bg-soft</code> background, mono font, header with language + Copy button, horizontal scroll (never wraps and never overflows the page)
Badge	Pill, small, <code>--bg-soft</code> fill, indigo text; <code>AiBadge</code> includes a tiny sparkle icon plus the word "AI-generated" (text present, not icon-only)
Toast	Bottom-centre, <code>aria-live="polite"</code> , auto-dismiss 4 s, errors persist until dismissed
Skeleton	Soft shimmer; static when reduced motion

14.8 Glow rules

Allowed	Not allowed
Hero halo (<code>--gradient-glow</code>)	Glow behind paragraphs of text

Allowed	Not allowed
Primary CTA, composer focus, input focus	Glow on every card
AI avatar, active conversation item	Pulsing/looping glow animations
404 illustration	Saturated neon (#00FFFF-style) colours

Glow intensity is one variable (`--glow-intensity`) so the whole product can be dimmed centrally.

14.9 Motion

Duration 120–360 ms with `--ease`. Allowed: hover lift, drawer slide, message fade-in (opacity + 4 px translate), typing dots. Not allowed: parallax, auto-playing large animations, bouncing. Under `prefers-reduced-motion`, transitions collapse to instant.

14.10 Design review checklist (used in Phase 11)

- ☐ Default theme is light, including on a dark-mode OS
- ☐ No hard-coded hex values in component CSS
- ☐ Every text/background pair verified $\geq 4.5:1$ (large text $\geq 3:1$)
- ☐ Glow appears only where §14.8 allows
- ☐ No text over the glow halo
- ☐ All interactive targets $\geq 44 \times 44$ px on mobile
- ☐ Focus ring visible on every interactive element
- ☐ Nothing depends on colour alone (strength meter, errors, badges have text/icons)

14.11 Theme system (UI-001, UI-010, FR-031)

- **Default:** `data-theme="light"`, ignoring `prefers-color-scheme` unless the user explicitly picks "System".
- **Storage:** `localStorage["neurochat-theme"] = light | dark | system`; when signed in, also saved to `settings.theme` so it follows the user between devices.
- **No flash:** a tiny inline script in `index.html` `<head>` sets `document.documentElement.dataset.theme` *before* React loads.
- **Implementation:** dark is a second token block `[data-theme="dark"] { ... }` that overrides the same variables (deep indigo surfaces, same accents lightened for contrast, glow slightly stronger). Components never mention "dark".
- **Priority:** dark mode is **MVP+**. If time is short, ship light only. The light-first requirement is never traded for it.

15 Authentication Architecture

15.1 Options compared (Guideline 24)

Option	How it works	Pros	Cons	Verdict
Server sessions (<code>express-session</code> + Mongo store)	Session ID cookie; data on server	Easy revocation	Extra store/config; more to explain	Good alternative
JWT in <code>localStorage</code>	Frontend stores token, sends <code>Authorization</code> header	Very common in tutorials	Any XSS can steal it; examiners often probe this	Rejected

Option	How it works	Pros	Cons	Verdict
JWT in httpOnly cookie	Server sets cookie; browser sends automatically	Not readable by JavaScript; simple; stateless verification	Needs CSRF thinking; cross-site cookie rules	Chosen (D2)
Access + refresh tokens	Short access token, long refresh token	Strong revocation story	Two tokens, rotation logic, more code	Overkill for scope

15.2 Token and cookie design

Item	Value
Algorithm	HS256, secret = <code>JWT_SECRET</code> (≥ 32 random bytes)
Claims	<code>sub</code> (user <code>_id</code>), <code>tv</code> (token version), <code>iat</code> , <code>exp</code>
Cookie name	<code>nc_token</code>
Flags	<code>HttpOnly</code> , <code>Secure</code> (production), <code>SameSite=Lax</code> , <code>Path=/</code>
Lifetime	Without "Remember me": session cookie, JWT <code>exp</code> 24 h. With it: <code>Max-Age</code> 30 days, JWT <code>exp</code> 30 d
Revocation	User has <code>tokenVersion</code> . Middleware compares <code>tv</code> to the user's current value. Incrementing it (password change, "log out everywhere") invalidates all old tokens. Ordinary logout clears the cookie

Viva answer to "how do you invalidate a JWT?": "Tokens carry a version number. Increasing the user's version makes every older token fail the check."

15.3 Registration flow

1. Client validates fields (UX only) and calls `POST /api/auth/register`.
2. Server validates with schema (SEC-005): name 2–60 chars, email format, password 8–72 chars with letter+number, confirm matches.
3. Normalise email (`trim().toLowerCase()`).
4. If email exists → `409 EMAIL_IN_USE` (distinct code if existing account is Google-only, so UI can suggest Google).
5. `bcrypt.hash(password, 12)`.
6. Create user (`authProviders:["local"]`, `emailVerified:false`).
7. Issue cookie, return user DTO (never the hash). The unique index on `email` is the final guard against race-condition duplicates; a duplicate-key error is mapped to 409.

15.4 Login flow

1. Validate body; normalise email.
2. Look up user by email. If missing, still run a dummy `bcrypt.compare` so response time does not reveal whether the email exists.
3. If user has no `passwordHash` (Google-only account): respond with the *same generic* failure.
4. Compare password. On mismatch → `401 INVALID_CREDENTIALS` ("Incorrect email or password").
5. Issue cookie (lifetime by `rememberMe`), return user DTO. Auth routes use a strict rate limit (SEC-008).

15.5 Authentication middleware (every protected route)

```

read cookie nc_token → missing? 401 UNAUTHENTICATED
verify signature + exp → invalid/expired? 401 (and clear cookie)
load user by sub (select minimal fields) → missing? 401
compare claims.tv with user.tokenVersion → mismatch? 401
attach req.user = { id, name, email } → next()

```

15.6 Logout

POST `/api/auth/logout` clears the cookie (same name/path/flags) and returns 204. Idempotent, works even with an expired session. The client clears `AuthContext`, cached conversation state, and calls Google's `disableAutoSelect()` if Google was used.

15.7 Client-side handling

- `AuthContext` bootstraps by calling `GET /api/auth/me` on load (`status: loading | authenticated | anonymous`). Route guards wait for `loading` to finish (no flash of login page).
- The API client (`services/api.js`) uses `credentials: 'include'` and a response interceptor: any `401` on a protected call (except the login/register calls themselves) triggers "session expired" → clear state → `navigate('/login?redirect=<current path>')` with a toast.
- Redirect targets are validated (`startsWith('/')` and not `//`) to prevent open-redirect abuse.

15.8 CSRF defence (SEC-012)

Cookie auth requires CSRF thought. Layers: (1) `SameSite=Lax` cookies are not sent on cross-site POSTs; (2) all state-changing endpoints use POST/PATCH/DELETE (never GET); (3) server checks `Origin` (or `Referer`) header against `CLIENT_URL` on those methods; (4) JSON-only bodies (`Content-Type: application/json`) which cross-site forms cannot send. Enough for academic level; explain it.

15.9 Password reset & email verification Planned

Not built for submission. Design kept so it can be added: `passwordResets` collection { `userId`, `tokenHash`, `expiresAt`, `usedAt` }, token = 32 random bytes sent by email (Nodemailer/SendGrid), store only the SHA-256 hash, expiry 30 min, single use, increments `tokenVersion` on success. Email verification uses the same pattern with `emailVerified`. **Consequence for MVP:** email ownership is *unverified* for local sign-ups, and §16.6 is designed around exactly that fact.

15.10 Known trade-offs (state them proudly in the viva)

- Registration reveals "email already registered" (needed for usability). Login does not leak.
- Stateless JWT cannot be individually revoked without `tokenVersion`; that check costs one small DB read, which we already need to load the user.

16

Google OAuth Architecture

16.1 Options compared

Option	Flow	Pros	Cons	Verdict
A. Google Identity Services, ID-token flow	Google button in browser → Google returns a signed ID token (JWT) to page → page sends it to backend → backend verifies with Google's keys → backend creates its own session	Real OpenID Connect; no client secret; no redirect handling; ~1 endpoint	Depends on Google's JS library and popup/cookie behaviour in browser	Chosen (D4)
B. Authorization-code flow (redirect, Passport/ <code>googleapis</code>)	Browser redirects to Google → Google redirects to backend <code>/callback?code=...</code> → backend exchanges code using client secret	Fully server-side; classic OAuth 2.0	Redirect URLs, state/PKCE, secret handling, more code	Valid; more to configure

Both are **real** authentication. A is chosen for simplicity, security (verification is server-side either way) and demonstrability.

Note on `GOOGLE_CLIENT_SECRET` : flow A does not use it. Keep it in `.env.example` as *optional*, only if you later switch to flow B. Never expose it to the frontend in any case.

16.2 Google Cloud configuration (step by step)

1. Google Cloud Console → **create a project** (e.g., `neurochat-academic`).
2. **APIs & Services → OAuth consent screen** (now labelled *Google Auth Platform* in newer console versions): User type **External**; app name `NeuroChat`; user-support email; developer contact email; add authorised domain when deployed.
3. **Scopes**: only `openid`, `email`, `profile` (non-sensitive). Do not request more.
4. **Publishing status**: while **Testing**, only accounts you list as *test users* can sign in. Either add every account you will use in the viva (yours, a spare, the examiner's if known), or set the app to **In production** (non-sensitive scopes normally do not require Google's app verification; check current Google requirements when you do it).
5. **Credentials → Create credentials → OAuth client ID → Application type: Web application**.
6. **Authorised JavaScript origins** (exact scheme + host + port; no paths, no trailing slash, no wildcards):
 - `http://localhost:5173` (Vite dev)
 - your production frontend origin, e.g. `https://neurochat.vercel.app`
7. **Authorised redirect URIs**: *not required* for flow A's JavaScript-callback mode. Only if you set the button to redirect UX mode, add the login endpoint URL Google will POST to (HTTPS in production).
8. Copy the **Client ID** (public). Put it in `VITE_GOOGLE_CLIENT_ID` (frontend) **and** `GOOGLE_CLIENT_ID` (backend). Changes to origins can take a few minutes to apply.
9. Common failure: `origin_mismatch` / "Error 400" → the running origin is not in the list exactly as typed (e.g., `127.0.0.1` vs `localhost`, or `http` vs `https`).

16.3 Frontend flow

User clicks "Continue with Google" (GoogleLogin component from `@react-oauth/google`, or GIS script)

- Google account chooser (popup / FedCM-based prompt)
- callback receives `{ credential }` ← this is the Google ID token (a JWT)
- frontend calls `POST /api/auth/google { credential, rememberMe }`
- on 200: `AuthContext.setUser(user)` → navigate to redirect target or `/chat`
- on error: show mapped message (§13.2)

The frontend **never** decodes the token to decide who the user is, and never sends name/email/picture as separate trusted fields.

16.4 Backend verification (`services/google.service.js`)

Using Google's official `google-auth-library` :

1. `client.verifyIdToken({ idToken, audience: GOOGLE_CLIENT_ID })` verifies: signature against Google's published public keys, `aud` equals **our** client ID (blocks tokens minted for other apps), `iss` is Google, `exp` not passed.
2. Extract from the verified payload: `sub` (stable Google user ID), `email`, `email_verified`, `name`, `picture`.
3. **Require** `email_verified === true`; otherwise `401`.
4. Everything the rest of the code uses comes from this verified payload, never from the request body.

16.5 User lookup / creation

Case	Detection	Action
New Google user	No user with this <code>googleId</code> , none with this email	Create user: <code>authProviders:["google"]</code> , <code>googleId=sub</code> , <code>emailVerified:true</code> , no <code>passwordHash</code> , <code>name</code> , <code>avatarUrl</code> . Issue session
Existing Google user	User found by <code>googleId</code>	Sign in. Optionally refresh <code>name</code> / <code>avatarUrl</code> . Issue session
Existing email account	No <code>googleId</code> match, but user with same email exists	See §16.6

Identity key for returning Google users is `googleId ( sub )`, **not email**, because emails can change while `sub` cannot.

16.6 Safe account linking (the important part)

Threat ("pre-hijacking"): because MVP does not verify email ownership at password sign-up, an attacker could register `victim@gmail.com` with a password they know. If the real victim later clicks "Continue with Google" and NeuroChat naïvely links the accounts, the attacker still knows the password and can enter the victim's account.

Rules (recommended, Option A):

1. Link automatically **only** when Google says `email_verified: true` (Google has proven the email belongs to the person in front of us).
2. If the existing local account has `emailVerified:false` , **discard its password hash** at linking time (`passwordHash = undefined`), bump `tokenVersion` (kicks any session an impostor might hold), set `googleId` , `emailVerified:true` , `authProviders:["google"]` .
3. If the existing local account has `emailVerified:true` (possible only after the **Planned** verification feature) → link and **keep** the password: both identities were proven.
4. Reverse direction: if someone tries to *register* with an email already owned by a Google account, refuse with `409 EMAIL_USES_GOOGLE` . Never let an unauthenticated visitor add a password to an existing account.
5. Result is always **one** user document per person; no duplicate identities.
6. **[MVP+ recovery path]:** a signed-in Google-only user can *set* a password from Settings (`PATCH /api/users/me/password`), which adds `"local"` to `authProviders` .

Alternative (Option B, stricter): never auto-link; return `409 ACCOUNT_EXISTS` and require the user to sign in with the password first and press "Connect Google" in Settings (two proofs). Safer for accounts with data but needs one more endpoint and screen. Documented so you can justify the choice in the viva.

16.7 Session and logout

After lookup/creation, the backend issues **its own** `nc_token` cookie (§15.2). Google's token is discarded; it is not stored. Logout follows §15.6.

16.8 Security considerations checklist

- ☐ Audience (`aud`) checked against our client ID
- ☐ Only `openid email profile` scopes
- ☐ `email_verified` enforced
- ☐ ID token never logged or stored; never sent to Gemini
- ☐ Rate limit on `/api/auth/google`
- ☐ HTTPS in production; exact origins in Google Console
- ☐ Frontend-provided profile fields ignored

- ☐ Profile picture URL stored as a string only; rendered with `referrerpolicy="no-referrer"` and an initials fallback if it fails to load

16.9 Testing without real Google

Automated tests mock `verifyIdToken` to return crafted payloads (new user, existing, unverified email, wrong audience → rejection). Manual tests use two real Google accounts once, per \$33/\$31.

16.10 30-second viva explanation

"The Google button gives my browser a signed proof of identity. My browser sends only that proof to my server. My server checks Google's signature and that it was issued for *my* app, reads the verified email and ID, finds or creates the user in MongoDB, and issues its own session cookie. The frontend is never trusted to tell the server who the user is."

17 AI Architecture

17.1 End-to-end message flow (AI-001)

```

React (Composer)
  → POST /api/conversations/:id/messages [cookie auth]
  → Express: rate limit → auth middleware → validation → ownership check
  → chat.controller (thin) → chat.service (business logic)
    1. save user message
    2. context.service: build bounded history
    3. ai.service → aiProvider (gemini.provider) → Gemini API
    4. validate/normalise the response
    5. sensitive-topic heuristic (metadata flag)
    6. save assistant message (+ metadata)
  ← 201 { userMessage, assistantMessage } → React renders
  
```

The browser never talks to Gemini. The API key exists only in backend environment variables.

17.2 Component responsibilities

Component	File (backend)	Responsibility
Controller	<code>controllers/message.controller.js</code>	HTTP only: read request, call service, shape response
Chat service	<code>services/chat.service.js</code>	Orchestrates a chat turn; owns persistence rules
Context builder	<code>services/context.service.js</code>	Chooses which messages to send (§19)
AI service	<code>services/ai.service.js</code>	Provider-neutral facade: <code>generateReply({ history, language })</code>
Gemini provider	<code>services/providers/gemini.provider.js</code>	The only file that imports the Gemini SDK
System prompt	<code>config/systemPrompt.js</code>	Central assistant instruction text
Sensitive-topic heuristic	<code>utils/sensitiveTopics.js</code>	Keyword/pattern check (AI-005)
Retrieval (empty)	<code>services/retrieval/index.js</code>	Planned extension point (§17.4)

17.3 Reliability design (addresses F1, F8)

NeuroChat cannot make a language model always correct. It can be **honest, transparent and recoverable**:

Measure	Behaviour	Req
System instruction	Tells the model: be accurate; say "I'm not sure" rather than invent; do not fabricate sources, quotes, or statistics; state that it has no live internet access when asked about current events; answer in the user's language; use Markdown	AI-003
Labelling	Every AI message carries an "AI-generated" badge	AI-004
Disclaimer	Persistent single line under the composer: "NeuroChat can make mistakes. Check important information."	AI-004
Verification nudge	Heuristic flags medical/legal/financial/safety prompts → note under the answer	AI-005
Response validation	Non-empty text; length cap; finish reason inspected; failures become error records, not blank bubbles	AI-007, AI-011
Recovery	Every failure is retryable	FR-022, AI-006
Citation-ready	<code>metadata.sources</code> field reserved; UI slot exists but hidden	AI-009

The heuristic in AI-005 is deliberately simple (multilingual keyword list such as *dose, symptom, medicine, lawsuit, tax, invest, loan, poison...*, including Hindi terms) and is documented as **best-effort**. It is not a safety system and must not be described as one.

17.4 Real-time information (F2): honest now, extensible later

Phase 1 (MVP): Gemini-based conversational AI only.

- No web access. The system instruction tells the model to say so; the landing page and Settings "About AI" say so.

Extension architecture (Phase 2, **Planned**, not implemented):

```
User message
→ [Router] needs current info? (rules or a cheap classifier) ← inserted between steps 1 and 2 of §17.1
→ Web retrieval / search provider (search API)
→ Source collection (title, URL, snippet, fetched text)
→ Source validation (allow/deny list, freshness, dedupe, size limits, strip instructions)
→ Context builder adds "SOURCES" block to the prompt
→ AI synthesis (Gemini) with instruction: use only provided sources, cite by number
→ Response + citations → stored in message.metadata.sources → UI "Sources" list
```

Where it fits in code: `services/retrieval/` exposes `search(query) → Source[]`; `chat.service` calls it only when the router says yes; `context.service` merges results; `message.metadata.sources` stores them. **No existing module changes shape.** **Risks to document:** prompt injection from web pages (treat fetched text as data), cost/quotas, latency, source quality. (An alternative is Gemini's built-in search grounding, if available on your tier; check current documentation. Even then, the response's sources should be stored and displayed.)

18 Gemini Integration

18.1 Configuration

Item	Approach
SDK	Google's official Gemini SDK for Node (@google/genai at time of writing; verify the current package and model names in Google AI documentation when you implement , as these change)
Key	<code>GEMINI_API_KEY</code> in backend <code>.env</code> only; loaded via <code>config/env.js</code> ; app refuses to start if missing

Item	Approach
Model	<code>GEMINI_MODEL</code> env variable holding a current fast, low-cost ("Flash"-class) model; never hard-coded in logic; saved in each message's <code>metadata.model</code>
Generation	<code>temperature</code> 0.7, <code>maxOutputTokens</code> ~2048 (tunable in <code>config/ai.js</code>)
System instruction	Passed via the SDK's system-instruction field, separate from user content
Streaming	Planned Phase 2 (Server-Sent Events). MVP is request/response

18.2 Request construction

```
contents = [ {role:"user"|"model", parts:[{text}] } ... ] ← from context.service (§19)
config = { systemInstruction, temperature, maxOutputTokens, (safety settings default) }
```

Language hint: if `settings.replyLanguage` $\equiv$ "auto", append one line to the system instruction ("Respond in Hindi (Devanagari) unless the user asks otherwise").

18.3 Timeout, retry, and concurrency

- **Timeout:** 30 s using `AbortController`. Timeout → `504 AI_TIMEOUT`.
- **Automatic retry:** at most **one** retry with short jittered delay, only for transient errors (network reset, 500/503). **No** automatic retry on 400 or 429 (retrying a quota error just wastes quota).
- **Concurrency guard:** a small in-process limiter (e.g., max 5 simultaneous Gemini calls) so a burst cannot exhaust the free-tier per-minute limit; excess requests wait briefly or get `429`.
- **Per-user limit:** 20 message requests/minute (SEC-008).

18.4 Error mapping (AI-006, AI-007, AI-011)

Situation	HTTP to client	<code>error.code</code>	Friendly UI message	Assistant message stored?
Invalid/missing server key, 401/403 from Google	502	<code>AI_CONFIG_ERROR</code>	"NeuroChat's AI service isn't available right now. Please try again later." (details only in server logs)	No
Quota/rate limit (429)	429	<code>AI_RATE_LIMITED</code>	"NeuroChat is busy right now. Wait a moment and retry."	No
Timeout	504	<code>AI_TIMEOUT</code>	"That took too long. Please retry."	No
Google 5xx / network	503	<code>AI_UNAVAILABLE</code>	"The AI service is temporarily unavailable. Please retry."	No
Empty/no candidates	502	<code>AI_EMPTY_RESPONSE</code>	"I couldn't produce an answer. Try rephrasing or retry."	No
Blocked by safety filter	422	<code>AI_BLOCKED</code>	"I can't help with that request. Try asking differently."	No
Truncated (max tokens)	201	none	Answer shown with note "This answer was cut short. Ask me to continue."	Yes, with <code>finishReason:"MAX_TOKENS"</code>
Malformed SDK response	502	<code>AI_BAD_RESPONSE</code>	Same as empty	No

Rule: the user's own message is **always saved first**, so it is never lost if the AI step fails. **No placeholder/error message is written to the database.** A conversation whose last message has role `user` is, by definition, "awaiting a reply", and the failed-send response returns that saved `userMessage` so the UI can show it with an inline error and Retry (§23.4).

18.5 Observability and privacy

Log: request ID, user ID, conversation ID, latency, model, status/error code, approximate token counts. **Do not log message content or API keys.** Add a privacy note: message text is sent to Google's Gemini API; **check the current Gemini API terms** regarding data use on free vs paid tiers and describe it truthfully on the Privacy page (§26.9).

18.6 Provider abstraction (AI-008)

`ai.service.generateReply(input)` returns a normalised object `{ text, finishReason, model, usage, sources? }`. Only `gemini.provider.js` knows Gemini's shapes. A second provider (or a mock for tests) implements the same function.

19

Conversation Context Architecture

19.1 Storage and identity

- Each **conversation** has a MongoDB `_id` (ObjectId) used in URLs (`/chat/:id`) and APIs. Each **message** references it via `conversationId` and carries `role` (`user` | `assistant`), `content`, `createdAt`.
- IDs from the client are always validated as ObjectIds and then checked for ownership (`{ _id, userId }`). Malformed → 404 (same response as "not yours").

19.2 Retrieval of history

Query used per turn (indexed on `{ conversationId, createdAt }`):

```
messages.find({ conversationId, userId })
  .sort({ createdAt: -1 }).limit(40)
```

then reversed into chronological order. Only the most recent 40 are even *fetched*.

19.3 Context construction algorithm (`context.service.js`)

```
INPUT: history (chronological, ≤40), newUserMessage
CONFIG: MAX_CONTEXT_MESSAGES = 20, MAX_CONTEXT_CHARS = 24000,
1. candidates = history + newUserMessage (all stored messages are valid turns).
2. Walk from newest to oldest; add each message while
   count ≤ MAX_CONTEXT_MESSAGES AND total chars ≤ MAX_CONTEXT_CHARS.
3. Stop at the first message that would exceed the budget (never skip-and-include-older; keep it contiguous).
4. Reverse to chronological order.
5. If the first item is an assistant message, drop it (contents should begin with a user turn).
6. Merge adjacent messages with the same role (happens when a failed AI step left two user turns).
7. Map roles: user→"user", assistant→"model".
8. Set truncated = (some older messages were left out).
OUTPUT: { contents, truncated }
```

- Why characters:** a cheap, deterministic proxy for tokens. English $\approx$ 4 chars/token; **Devanagari uses noticeably more tokens per character**, so the budget is deliberately conservative. **MVP+** may call the SDK's token-counting method for exactness.
- The input limit of 4,000 characters guarantees the newest message always fits.
- If `truncated` is true, the response includes `meta.contextTruncated:true` and the UI shows a small note: "Very long conversation: NeuroChat may not remember the earliest messages." (honest, F8).

19.4 Strategies compared

Strategy	Description	Pros	Cons	Verdict
Send full history always	Everything, every time	Perfect memory	Cost, latency and limits grow without bound; violates the requirement	Rejected
Sliding window with budget	Last N messages within size cap	Simple, bounded, easy to explain	Forgets old details	MVP (D6)
Rolling summary	Periodically summarise old messages into <code>conversation.summary</code> , send summary + recent window	Keeps gist of long chats	Extra AI calls; summary drift	Planned : fields reserved (<code>summary</code> , <code>summarizedUntil</code>)
Retrieval over old messages	Embed and fetch relevant old messages	Best long-term recall	Embeddings, vector search	Future

19.5 Worked example

```

Turn 1  user: "What is Java?"                → contents=[user1]
      model: explains Java                    → stored
Turn 2  user: "What are its advantages?"      → contents=[user1, model1, user2]
      model sees Java in model1 → answers about Java

```

The proof for the viva: open the two `messages` documents in Compass, and show that turn 2's request would contain turn 1.

19.6 Edge cases

- **Two tabs, same conversation:** last-write ordering by `createdAt` ; acceptable at this scope.
- **Failed turn:** user message saved, no assistant message; Retry generates the missing reply from the same history. If the user instead sends another message first, two consecutive user turns exist and step 6 merges them.
- **Deleted conversation mid-request:** the save step fails cleanly with 404.
- **Language switching mid-chat:** no special handling; context is language-agnostic.

20 Multilingual Architecture

Honest scope: NeuroChat *supports* multilingual use through (1) Unicode-safe plumbing, (2) a translated interface, and (3) the language ability of the Gemini model. It does **not** include its own translation engine, and reply quality in a given language depends on the model.

Layer	Design	Status
Storage	MongoDB stores UTF-8. JSON APIs use <code>application/json; charset=utf-8</code> . No ASCII-only regexes, <code>slice()</code> on bytes, or <code>\w</code> on names/titles	MVP (NFR-007)
UI translation	<code>react-i18next</code> with <code>locales/en.json</code> , <code>locales/hi.json</code> . All user-visible strings via <code>t('key')</code> , none hard-coded in components. Dates/numbers via <code>Intl</code>	MVP English + Hindi
Marathi and others	Add <code>mr.json</code> and one dropdown entry; nothing else changes	Planned
Language preference	<code>settings.uiLanguage</code> (<code>en</code> / <code>hi</code>) and <code>settings.replyLanguage</code> (<code>auto</code> / <code>en</code> / <code>hi</code>). Stored per user, cached in <code>localStorage</code> for instant load	MVP

Layer	Design	Status
Reply language	<code>auto</code> : system instruction says "reply in the language the user writes in". Explicit language: extra instruction line (§18.2)	MVP
Fonts	<code>Noto Sans Devanagari</code> in font stacks (§14.6); <code><html lang></code> updated when language changes	MVP
Text handling	Title truncation by characters using <code>Array.from</code> (or <code>Intl.Segmenter</code> for grapheme clusters); MongoDB text index uses <code>default_language: "none"</code> (MongoDB's built-in stemming does not cover Hindi/Marathi, so tokenise on whitespace)	MVP / MVP+
Voice language	Recogniser <code>lang</code> set from preference (<code>en-IN</code> , <code>hi-IN</code>)	MVP+
Mixed/Hinglish input	Passed through unchanged; the model handles it	Automatic
RTL scripts	Not needed now; using CSS logical properties (<code>margin-inline</code> , <code>padding-inline</code>) keeps the door open	Future

Do not: assume English in validation (names may contain Devanagari), sort or search with English-only rules, or translate the AI's answers on the client.

Test strings (keep in the test plan): नमस्ते, मुझे जावा समझाओ, मराठीत सांग, Café naïve, an emoji, and a 4,000-character Devanagari paragraph.

21 Voice Architecture

Voice is a **modular, optional layer** that uses browser APIs. The chat works fully without it.

21.1 Classification

Capability	Technology	Status
Voice input (dictation into the text box)	Web Speech API <code>SpeechRecognition</code> (<code>webkitSpeechRecognition</code> in Chromium browsers)	MVP+
Voice output (read a response aloud)	Web Speech API <code>speechSynthesis</code>	MVP+
Hands-free continuous voice conversation (auto-send, auto-listen)	Same APIs plus turn-taking logic	Future
Server-side speech-to-text/text-to-speech, or a model's native audio mode	Cloud speech services or Gemini audio features	Future

21.2 Data flow

Voice input : Mic button → browser asks microphone permission → `SpeechRecognition` (`lang` from preference) → interim + final transcript → written into the Composer textarea → user reviews → presses Send
 Voice output: AI message → "Listen" button → strip Markdown to plain text (code blocks read as "code block omitted")
 → `speechSynthesis.speak(utterance, lang)` → Stop button / stops on navigation

The transcript is only text in the textarea; **the backend does not know voice was used** and receives ordinary message text.

21.3 Module boundaries

`hooks/useSpeechRecognition.js`, `hooks/useSpeechSynthesis.js`, `components/voice/MicButton.jsx`, `components/voice/ListenButton.jsx`. Feature detection at startup: unsupported → components render nothing (no broken buttons).

21.4 Known limits (write these in the report)

- Speech recognition is not available in all browsers (notably Firefox), needs HTTPS or `localhost`, and needs microphone permission.
- In Chrome/Edge, recognition audio is typically processed by the browser vendor's online service. Tell users in the Privacy page and the first mic use.
- Hindi/Marathi synthesis voices depend on the operating system's installed voices.
- Recognition accuracy varies with accent and noise; hence "review before sending".

21.5 States and accessibility

Mic states: idle → requesting permission → listening (visible + announced via `aria-live`) → processing → idle, plus denied/unsupported/no-speech errors with plain messages. Buttons have text labels or `aria-label`; voice is an *addition* to keyboard input, never a replacement.

22 Database Design

22.1 Design decisions

- Three collections: **users**, **conversations**, **messages**. Referenced, not embedded: a conversation could hold thousands of messages, an embedded array would grow unbounded (MongoDB's 16 MB document limit), could not be paginated efficiently, and would make search awkward.
- `userId` is stored **on messages too (D8)**. This duplicates one 12-byte ObjectId (not the large content) and lets every message query be scoped by owner in one step and enables per-user message search without joins.
- Mongoose schemas add validation and defaults; the database is `neurochat`.

22.2 Relationships

```
User (1) —< Conversation (many)  conversations.userId → users._id
Conversation (1) —< Message (many) messages.conversationId → conversations._id
User (1) —< Message (many)       messages.userId → users._id (denormalised for authorisation/search)
```

22.3 users

Field	Type	Rules
<code>_id</code>	ObjectId	auto
<code>name</code>	String	required, trimmed, 2–60
<code>email</code>	String	required, lower-cased , unique
<code>passwordHash</code>	String	optional (absent for Google-only); <code>select:false</code> ; never returned by API
<code>authProviders</code>	[String]	subset of <code>["local", "google"]</code> (equivalent of <code>authProvider</code> , allows linking)
<code>googleId</code>	String	optional; unique sparse ; Google <code>sub</code>
<code>emailVerified</code>	Boolean	default false (true for Google-created)
<code>avatarUrl</code>	String	optional
<code>settings</code>	Object	<code>{ uiLanguage:"en", replyLanguage:"auto", theme:"light", onboardingCompleted:false }</code>
<code>tokenVersion</code>	Number	default 0 (§15.2)
<code>lastLoginAt</code>	Date	set on login

Field	Type	Rules
<code>createdAt</code> , <code>updatedAt</code>	Date	Mongoose timestamps

22.4 conversations

Field	Type	Rules
<code>_id</code>	ObjectId	auto
<code>userId</code>	ObjectId	required, ref users, indexed
<code>title</code>	String	default "New chat", 1-100; auto-set from first message
<code>summary</code>	String	Planned rolling summary (§19.4)
<code>summarizedUntil</code>	ObjectId	Planned
<code>createdAt</code> , <code>updatedAt</code>	Date	<code>updatedAt</code> is bumped whenever a message is added (drives sidebar order)

22.5 messages

Field	Type	Rules
<code>_id</code>	ObjectId	auto (monotonic-ish; also gives stable pagination cursor)
<code>conversationId</code>	ObjectId	required, ref conversations
<code>userId</code>	ObjectId	required (owner; copied from conversation)
<code>role</code>	String	enum <code>user</code> <code>assistant</code>
<code>content</code>	String	required; user $\leq$ 4,000 chars (validator), assistant $\leq$ 20,000
<code>createdAt</code>	Date	immutable
<code>metadata</code>	Object	assistant only, all optional: <code>model</code> , <code>finishReason</code> , <code>latencyMs</code> , <code>usage{promptTokens,completionTokens}</code> , <code>sensitiveTopic:Boolean</code> , <code>contextTruncated:Boolean</code> , <code>sources:[{title,url}]</code> (Planned)

22.6 Indexes

Collection	Index	Purpose
users	<code>{ email: 1 }</code> unique	login, duplicate prevention
users	<code>{ googleId: 1 }</code> unique, sparse	Google login lookup (sparse so many users without <code>googleId</code> are allowed)
conversations	<code>{ userId: 1, updatedAt: -1, _id: -1 }</code>	sidebar list and cursor pagination
conversations	<code>{ userId: 1, title: 1 }</code>	optional, helps title search
messages	<code>{ conversationId: 1, _id: 1 }</code>	load and page history in order
messages	<code>{ userId: 1, content: "text" }</code> with <code>default_language:"none"</code>	MVP+ message-content search

Verify with `explain()` that list/history queries use these indexes (a good viva point).

22.7 Referential integrity and deletion

MongoDB does not enforce foreign keys, so the application does. **Delete conversation:** verify ownership → `messages.deleteMany({ conversationId, userId })` → `conversations.deleteOne({ _id, userId })`

(idempotent; a MongoDB transaction may wrap both if using Atlas). **Delete account** **Planned** cascades the same way.

22.8 Sample documents (for report and viva)

```
// users
{ "_id": "66f0...a1", "name": "Aditya", "email": "aditya@example.com",
  "passwordHash": "$2b$12$Kx...(60 chars)", "authProviders": ["local"], "emailVerified": false,
  "settings": { "uiLanguage": "en", "replyLanguage": "auto", "theme": "light", "onboardingCompleted": true },
  "tokenVersion": 0, "createdAt": "2026-10-01T09:00:00Z", "updatedAt": "2026-10-01T09:00:00Z" }

// conversations
{ "_id": "66f0...c7", "userId": "66f0...a1", "title": "What is Java?",
  "createdAt": "2026-10-01T09:05:00Z", "updatedAt": "2026-10-01T09:06:12Z" }

// messages
{ "_id": "66f0...d1", "conversationId": "66f0...c7", "userId": "66f0...a1", "role": "user",
  "content": "What is Java?", "createdAt": "2026-10-01T09:05:00Z" }
{ "_id": "66f0...d2", "conversationId": "66f0...c7", "userId": "66f0...a1", "role": "assistant",
  "content": "Java is a ...", "createdAt": "2026-10-01T09:05:04Z",
  "metadata": { "model": "<GEMINI_MODEL>", "finishReason": "STOP", "latencyMs": 2310 } }
```

22.9 Using MongoDB Compass in the viva

1. **Connect** with the connection string (Atlas: *Connect* → *Compass*; local: `mongodb://localhost:27017`). Use a **read-only database user** for the demo and never show the password on screen.
2. Open database `neurochat`. Point out the three collections.
3. **users**: filter `{ "email": "..." }`; show `passwordHash` is a bcrypt hash (starts `$2b$12$`), not the password; show `googleId` on a Google user.
4. **conversations**: filter `{ "userId": ObjectId("...") }`; show `title` and `updatedAt`.
5. **messages**: filter `{ "conversationId": ObjectId("...") }`, sort `{ _id: 1 }`; show alternating roles and the Hindi test message displaying correctly.
6. Open the **Indexes** tab to show the indexes in §22.6; optionally the **Explain Plan** tab on the history query.
7. Delete a conversation in the app, hit refresh in Compass, and show its messages are gone.

23 API Specification

23.1 Conventions

- **Base path**: `/api`. JSON only (`Content-Type: application/json; charset=utf-8`). Authentication is the `nc_token` cookie; clients send `credentials: 'include'`.
- **Success envelope**: `{ "success": true, "data": { ... }, "meta": { ... } }`
- **Error envelope**: `{ "success": false, "error": { "code": "STRING", "message": "Human-readable", "fields": { "email": "..." }, "retryable": false, "requestId": "..." } }`
- **Never** returned: stack traces, SQL/Mongo errors, secrets, password hashes.
- **Ownership**: a resource that does not exist *or belongs to someone else* returns the same `404 NOT_FOUND` (does not reveal existence).
- **IDs**: 24-char hex ObjectIds; malformed → `404 NOT_FOUND`.

User DTO: `{ id, name, email, avatarUrl, authProviders, settings, createdAt }` **Conversation DTO**: `{ id, title, createdAt, updatedAt }` **Message DTO**: `{ id, conversationId, role, content, createdAt, metadata? }`

Common status codes: 200 OK · 201 Created · 204 No Content · 400 validation · 401 unauthenticated · 403 forbidden (bad Origin/CSRF) · 404 not found · 409 conflict · 413 payload too large · 422 AI blocked · 429 rate limited · 500 internal · 502 upstream/AI error · 503 AI unavailable · 504 AI timeout.

Rate limits: auth endpoints 10 requests / 15 min / IP; message endpoints 20 / min / user; all other authenticated 120 / min / user. Responses include `Retry-After` on 429.

23.2 Authentication

A1 · `POST /api/auth/register` · Auth: none

- Body: `{ name, email, password, confirmPassword, rememberMe? }`
- Validation: name 2–60 (Unicode letters/spaces/.'-); valid email ≤ 254 ; password 8–72 with ≥ 1 letter and ≥ 1 digit; `confirmPassword` $\equiv$ `password`.
- Success 201: `{ data: { user } }` + `Set-Cookie: nc_token`.
- Errors: 400 `VALIDATION_ERROR` (with `fields`), 409 `EMAIL_IN_USE`, 409 `EMAIL_USES_GOOGLE`, 429 `RATE_LIMITED`.

A2 · `POST /api/auth/login` · Auth: none

- Body: `{ email, password, rememberMe? }`. Validation: email format; password non-empty string ≤ 72 .
- Success 200: `{ data: { user } }` + cookie.
- Errors: 400 `VALIDATION_ERROR`, 401 `INVALID_CREDENTIALS` (identical for unknown email, wrong password, Google-only account), 429.

A3 · `POST /api/auth/google` · Auth: none

- Body: `{ credential, rememberMe? }` (`credential` = Google ID token, string ≤ 4 KB).
- Success 200: `{ data: { user, isNewUser } }` + cookie.
- Errors: 400 `VALIDATION_ERROR`, 401 `GOOGLE_TOKEN_INVALID` (bad signature/audience/expiry), 401 `GOOGLE_EMAIL_UNVERIFIED`, 429, 502 `GOOGLE_UNAVAILABLE` (could not fetch Google's keys).

A4 · `POST /api/auth/logout` · Auth: optional

- Body: none. Success 204 + cookie cleared. Never fails because of an expired session.

A5 · `GET /api/auth/me` · Auth: required

- Success 200: `{ data: { user } }`. Errors: 401 `UNAUTHENTICATED`. Used to bootstrap the session on page load.

23.3 Conversations

C1 · `POST /api/conversations` · Auth: required

- Body: `{ title? }` (default "New chat", 1–100 chars). The client calls this at the moment the user sends their *first* message, not when "New chat" is clicked (§11.3).
- Success 201: `{ data: { conversation } }`. Errors: 400, 401.

C2 · `GET /api/conversations` · Auth: required

- Query: `limit` (1–50, default 20), `cursor` (opaque, from previous `meta.nextCursor`), `search` (≤ 100 chars; escaped, case-insensitive title match), `searchIn=messages` **MVP+**.
- Success 200: `{ data: { conversations: [ConversationDTO] }, meta: { nextCursor: string|null } }`, sorted `updatedAt` desc, only the caller's.
- Errors: 400 (bad limit/cursor), 401.

C3 · `GET /api/conversations/:id` · Auth: required

- Success 200: `{ data: { conversation } }`. Errors: 401, 404.

C4 · `PATCH /api/conversations/:id` · Auth: required

- Body: `{ title }`, trimmed 1–100. Success `200: { data: { conversation } }`. Errors: `400`, `401`, `404`.

C5 · `DELETE /api/conversations/:id` · Auth: required

- Success `204` (conversation and all its messages removed). Errors: `401`, `404`. Client shows a confirmation dialog first.

23.4 Messages

M1 · `GET /api/conversations/:id/messages` · Auth: required

- Query: `limit` (1–100, default 50), `before` (message ID; return messages older than it).
- Success `200: { data: { messages: [MessageDTO] (chronological) }, meta: { hasMore } }`. Errors: `400`, `401`, `404`.

M2 · `POST /api/conversations/:id/messages` · Auth: required · Rate: 20/min/user

- Body: `{ content, language? }` — `content` trimmed, 1–4,000 chars, must be a string (objects rejected, SEC-013); `language` optional override (`en` / `hi`).
- Server steps: validate → ownership → save user message (and, if this is the conversation's first message, set the title) → build context (§19) → call Gemini (§18) → save assistant message → bump `updatedAt`.
- Success `201: { data: { userMessage, assistantMessage, conversation }, meta: { contextTruncated, sensitiveTopic } }`.
- AI failure** (status per §18.4: 422/429/502/503/504): body is `{ success:false, error:{ code, message, retryable:true }, data:{ userMessage, conversation } }`. The **user message is already saved** and returned, so the UI can keep showing it with an inline error and Retry.
- Other errors: `400`, `401`, `404`, `413`, `429`.

M3 · `POST /api/conversations/:id/messages/retry` · Auth: required · Rate: same as M2

- Body: none. Behaviour: if the last message is a **user** message (previous attempt failed) → generate the missing reply. If the last message is an **assistant** message → *regenerate*: generate a new reply from the same history and replace the old one only after the new one succeeds (a failure never destroys the existing answer).
- Success `200: { data: { assistantMessage, conversation } }`.
- Errors: `404` (also when conversation is empty), `409 NOTHING_TO_RETRY`, AI errors as M2, `401`, `429`.

23.5 User / profile

U1 · `PATCH /api/users/me` · Auth: required

- Body (any subset): `{ name?, settings?: { uiLanguage?: "en"|"hi", replyLanguage?: "auto"|"en"|"hi", theme?: "light"|"dark"|"system", onboardingCompleted?: boolean } }`. Unknown keys rejected. Email cannot be changed.
- Success `200: { data: { user } }`. Errors: `400`, `401`.
- Profile is read with A5 (`GET /api/auth/me`), which returns the full user DTO including settings.

U2 · `PATCH /api/users/me/password` · Auth: required · **MVP+**

- Body: `{ currentPassword?, newPassword }` (`currentPassword` required if the account already has a password; a Google-only user may set one). Success `204`; increments `tokenVersion` and re-issues the cookie. Errors: `400`, `401`, `401 INVALID_CREDENTIALS`.

U3 · `DELETE /api/users/me` · **Planned** (account and data deletion).

23.6 Utility

S1 • GET `/api/health` • Auth: none. 200 { status:"ok", db:"up" } (or 503 if DB unreachable). Used for deployment checks and uptime pings that reduce free-tier cold starts.

23.7 Endpoint summary

ID	Method	Path	Auth	Status
A1	POST	<code>/api/auth/register</code>	No	MVP
A2	POST	<code>/api/auth/login</code>	No	MVP
A3	POST	<code>/api/auth/google</code>	No	MVP
A4	POST	<code>/api/auth/logout</code>	Optional	MVP
A5	GET	<code>/api/auth/me</code>	Yes	MVP
C1	POST	<code>/api/conversations</code>	Yes	MVP
C2	GET	<code>/api/conversations</code>	Yes	MVP
C3	GET	<code>/api/conversations/:id</code>	Yes	MVP
C4	PATCH	<code>/api/conversations/:id</code>	Yes	MVP
C5	DELETE	<code>/api/conversations/:id</code>	Yes	MVP
M1	GET	<code>/api/conversations/:id/messages</code>	Yes	MVP
M2	POST	<code>/api/conversations/:id/messages</code>	Yes	MVP
M3	POST	<code>/api/conversations/:id/messages/retry</code>	Yes	MVP
U1	PATCH	<code>/api/users/me</code>	Yes	MVP
U2	PATCH	<code>/api/users/me/password</code>	Yes	MVP+
U3	DELETE	<code>/api/users/me</code>	Yes	Planned
S1	GET	<code>/api/health</code>	No	MVP

24 Frontend Architecture

24.1 Technology choices

Concern	Choice	Alternatives considered	Reason
Build tool	Vite + React 18+	Create React App (deprecated)	Fast, current standard
Language	JavaScript (ES modules)	TypeScript	Lower cognitive load for a first full-stack build. TypeScript is a valid upgrade; if chosen, use it from Phase 0, never half-migrated
Routing	React Router	Next.js	SPA is enough; API is a separate backend as required
State	React Context + custom hooks (<code>AuthContext</code> , <code>ThemeContext</code> , <code>ChatContext</code>)	Redux Toolkit, TanStack Query	Enough for this scale and easy to explain. TanStack Query is a reasonable upgrade for caching
HTTP	Thin <code>fetch</code> wrapper (<code>services/api.js</code>)	Axios	One less dependency; handles cookies, JSON, error envelope, 401

Concern	Choice	Alternatives considered	Reason
Styling	CSS Modules + design tokens (CSS variables)	Tailwind, styled-components	Central theming is the requirement; plain CSS is fully explainable
Markdown	<code>react-markdown</code> + <code>remark-gfm</code> + <code>rehype-sanitize</code> ; no <code>rehype-raw</code>	<code>dangerouslySetInnerHTML</code> + <code>DOMPurify</code>	Renders to React elements, so no raw HTML ever enters the DOM
Code highlighting	Lazy-loaded highlighter (e.g., <code>rehype-highlight</code>)	none	Loaded only when a code block first appears
i18n	<code>react-i18next</code>	hand-rolled	Standard, lazy-loads locales
Google	<code>@react-oauth/google</code> (wraps Google Identity Services)	raw GIS script	Less boilerplate, still real GIS
Icons	<code>lucide-react</code> or inline SVG	icon fonts	Tree-shakable
Tests	Vitest + React Testing Library; Playwright optional	Jest	Native to Vite

24.2 Folder responsibilities

```

frontend/src/
├─ main.jsx           App bootstrap (providers, router)
├─ App.jsx           Route table
├─ assets/           Logo SVGs, illustrations, fonts (local)
├─ components/
│  ├─ ui/            Design-system atoms: Button, TextField, PasswordField, Modal, Toast, Badge, Skeleton...
│  ├─ layout/        Navbar, Footer, Sidebar, TopBar, Drawer
│  ├─ landing/       Hero, HeroVisual, FeatureCard, StepCard, UseCases, TrustPanel
│  ├─ auth/          AuthCard, GoogleSignInButton, PasswordStrengthMeter
│  └─ chat/          MessageList, MessageBubble, MarkdownRenderer, CodeBlock, Composer, TypingIndicator,
ErrorBanner, VerificationNote, WelcomeState
├─ conversations/    ConversationList, ConversationItem, SearchBox, ConfirmDialog
├─ onboarding/       OnboardingModal
├─ voice/            MicButton, ListenButton [MVP+]
├─ common/           ErrorBoundary, ProtectedRoute, ScrollToTop
├─ pages/            LandingPage, LoginPage, RegisterPage, ChatPage, SettingsPage, PrivacyPage, TermsPage,
NotFoundPage
├─ layouts/          PublicLayout, AuthLayout, AppLayout (sidebar + main)
├─ hooks/            useAuth, useConversations, useMessages, useDebounce, useTheme, useMediaQuery, useSpeech*...
├─ context/          AuthContext, ThemeContext, ChatContext
├─ services/         api.js (fetch wrapper), authService, conversationService, messageService, userService
├─ i18n/             config + locales/en.json, hi.json
├─ utils/            validators, formatters (dates), sensitive-input helpers, constants
└─ styles/           tokens.css, base.css (reset, typography), themes.css (dark), utilities.css

```

Rules: *Pages* compose components and call hooks; *components* are presentational and reusable; *hooks* hold behaviour and state; *services* are the only place that knows URLs; *styles/tokens.css* is the only place with raw colour values. No component over ~200 lines without splitting.

24.3 Routing and guards

`ProtectedRoute` waits for `AuthContext.status` $\neq$ `"loading"`, redirects anonymous users to `/login?redirect=...`; `PublicOnlyRoute` sends signed-in users away from login/register. Route-level `React.lazy` for `SettingsPage`, legal pages and the Markdown/code-highlight bundle.

24.4 Chat state machine (per send)

```

idle → sending (optimistic user bubble, typing indicator, composer disabled)
      → success (append assistant message, re-enable composer, scroll)
      → failed (keep user bubble, show ErrorBanner + Retry; composer re-enabled)
failed → retrying → success | failed

```

`useMessages(conversationId)` owns this. The composer is disabled *only* while a request is in flight for that conversation. Component-level `AbortController` cancels obsolete history loads when the user switches conversations quickly.

24.5 Environment (frontend, public values only)

`VITE_API_BASE_URL` (`/api` when using the proxy) and `VITE_GOOGLE_CLIENT_ID`. **Anything with `VITE_` is public in the browser bundle.** Never put secrets there.

25 Backend Architecture

25.1 Technology choices

Concern	Choice	Note
Runtime / framework	Node.js (current LTS) + Express	Express 5 handles async errors natively; keep an <code>asyncHandler</code> wrapper so code works on Express 4 too
ODM	Mongoose	Schemas + validation, easy to explain
Validation	zod	One schema per endpoint; strips unknown keys; type-checks (blocks NoSQL operator injection)
Passwords	bcryptjs (cost 12)	Pure JS, no native build problems on Windows/Render; <code>bcrypt</code> or Argon2 are equally valid
Tokens	<code>jsonwebtoken</code> , <code>cookie-parser</code>	
Google	<code>google-auth-library</code>	Official ID-token verification
Gemini	Official Gemini Node SDK	Only in <code>gemini.provider.js</code>
Hardening	<code>helmet</code> , <code>cors</code> , <code>express-rate-limit</code>	
Logging	<code>pino</code> (JSON) or <code>morgan</code> (dev)	No message content in logs
Testing	Vitest/Jest + Supertest + <code>mongodb-memory-server</code>	Real routes, in-memory DB, mocked AI

25.2 Layering

```

HTTP request
→ routes/      map URL+method → middleware chain → controller
→ middleware/  auth, validation, rate limit, origin check, error handling
→ controllers/ HTTP concerns only (req → service call → res)
→ services/    business logic (auth, chat, context, ai, google, title)
→ models/      Mongoose schemas (data access)
→ MongoDB

services/providers/gemini.provider.js → Gemini API
  
```

A controller never touches Mongoose or Gemini directly; a service never touches `req` / `res`.

25.3 Middleware order (matters)

1. `requestId` → 2. `helmet` → 3. `cors` (allow-list, `credentials:true`) → 4. `express.json({ limit: "32kb" })` → 5. `cookieParser` → 6. `originCheck` for POST/PATCH/DELETE → 7. routes (with per-route rate limiters, `validate(schema)`, `requireAuth`) → 8. `notFound` → 9. `errorHandler` (last).

25.4 Error handling design

- `utils/ApiError.js`: `new ApiError(status, code, message, fields?, retryable?)`.

- `errorHandler` converts: `ApiError` → its envelope; `ZodError` → 400 `VALIDATION_ERROR` with `fields`; Mongo duplicate key (11000) → 409; `CastError` /bad ObjectId → 404; JWT errors → 401; upstream AI errors → per §18.4; **anything else → generic 500** ("Something went wrong") with details logged server-side only (`NODE_ENV=production` never echoes error text or stack).
- Unhandled rejections/uncaught exceptions are logged and the process exits (platform restarts it).

25.5 Configuration and startup

`config/env.js` validates all environment variables with zod at boot and **fails fast** with a clear message if one is missing (e.g., no `JWT_SECRET`). `config/db.js` connects to MongoDB and exits on failure. `app.js` builds the Express app (importable by tests); `server.js` only starts listening.

25.6 Backend folder responsibilities

Listed in §33.2.

26 Security Architecture

26.1 Threat and control summary

Threat	Control	Req
Stolen passwords from DB leak	bcrypt hashes, cost 12, <code>select:false</code>	SEC-001
Token theft via XSS	httpOnly cookie; sanitised Markdown; no <code>dangerouslySetInnerHTML</code> ; CSP via helmet	SEC-002, SEC-009, SEC-011
CSRF	SameSite=Lax + Origin check + JSON-only + non-GET mutations	SEC-012
Forged/expired/tampered token	Signature + expiry + <code>tokenVersion</code> check on every request	SEC-003
IDOR (change an ID to read another user's chat)	Every query includes <code>userId</code> from <code>req.user</code> , never from client; 404 for foreign IDs; automated test	SEC-004
Brute-force login	Strict per-IP limiter; generic errors; bcrypt cost	SEC-008, SEC-012
NoSQL injection	zod type validation; never spread <code>req.body</code> into queries; escaped regex for search	SEC-005, SEC-013
Fake Google identity	Server-side <code>verifyIdToken</code> , audience check, <code>email_verified</code>	SEC-010
Account pre-hijacking	§16.6 linking rules	SEC-010
API key leak	Backend env only; <code>.env</code> git-ignored; <code>.env.example</code> ; fail-fast; bundle scan	SEC-006, SEC-014, AI-001
Cross-origin abuse	CORS allow-list of exact origins	SEC-007
Abuse of AI quota	Per-user limiter, concurrency guard, input length cap	SEC-008
Oversized payloads	32 KB body limit; 4,000-char message cap	SEC-005
Open redirect after login	Redirect target must start with <code>/</code> and not <code>//</code>	SEC-005
Info leakage in errors	Central error handler; no stacks in production	§27
Vulnerable dependencies	<code>npm audit</code> before submission; lock files committed	NFR-004

26.2 Secrets

- Only in environment variables (`.env` locally; host dashboard in production).
- `.gitignore` includes `.env*` (except `.env.example`) from the **first commit**. If a key is ever committed, **rotate it immediately**; deleting the file from Git is not enough because history keeps it.

- Consider a pre-commit secret scan (e.g., gitleaks).

26.3 CORS and cookies

`origin` = exact `CLIENT_URL` (no `*`, because credentials are used). With the same-origin proxy (D3), CORS is rarely exercised in production but stays configured for local development (`5173` → `5000`).

26.4 Safe Markdown pipeline

All text → `react-markdown` (produces React elements) → `remark-gfm` → `rehype-sanitize` with a strict schema (allow: p, h1–h4, ul/ol/li, code, pre, em, strong, blockquote, table family, a). Additional rules: **images disabled** (an image URL can leak that a user viewed a message); links get `target="_blank"` `rel="noopener noreferrer nofollow"`; only `http`, `https`, `mailto` schemes; no HTML passthrough. User messages are rendered as plain text (or the same pipeline), never as HTML.

26.5 Security headers

`helmet` defaults, plus a Content-Security-Policy on the frontend host that permits only own scripts/styles/fonts and Google's Identity Services origins. Test with the deployed site; a too-strict CSP will silently break the Google button.

26.6 Logging and monitoring

Request ID on every log line and returned in error responses (so a screenshot of an error leads you to the log). Log auth failures (without credentials) and rate-limit hits.

26.7 Authorisation matrix

Resource	Anonymous	Owner	Other user
Landing, legal pages	✓	✓	✓
Auth endpoints	✓	✓	✓
Own profile	✗ (401)	✓	n/a
Conversation / messages	✗ (401)	✓	✗ (404)

26.8 Security test evidence

Each control above has a test in §31.3. Save screenshots (network tab without key; Compass showing hash; 404 for foreign ID) for the report.

26.9 Privacy statement content (NFR-010)

The Privacy page must truthfully state:

- **What is stored:** name, email, hashed password (if used), Google ID and picture URL (if used), settings, conversations and messages.
- **What is sent to third parties:** message text (and recent conversation context) is sent to **Google's Gemini API** to produce answers; Google Sign-In shares name/email/picture with NeuroChat; browser voice recognition may send audio to the browser vendor. *Check Google's current Gemini API terms about how data may be used and retained, and describe it accurately, especially for the free tier.*
- **Advice to users:** do not enter passwords, ID numbers, or highly sensitive personal information.
- **Deletion:** users can delete conversations now; account deletion is **Planned**. State this, do not promise what is not built.
- **Cookies:** one essential session cookie; no advertising/analytics cookies (if you add analytics later, update this).

27 Error Handling

27.1 State matrix (every operation)

State	Rule	Example
Loading	Visible within 100 ms; skeletons for lists, spinner on buttons, typing indicator for AI; controls disabled to prevent duplicate submits	Login button spinner; sidebar skeleton
Success	Confirm where the result isn't obvious (toast); otherwise the UI change is the confirmation	"Conversation renamed" toast; new message appears
Validation error	Field-level message beside the field + summary for screen readers; says <i>what to fix</i>	"Password must be at least 8 characters and include a number"
Authentication error	Generic, never reveals which part failed; no backend details	"Incorrect email or password"
Gemini/AI error	Friendly message from §18.4 inline under the user's message + Retry ; user's text is preserved	"NeuroChat is busy right now. Wait a moment and retry."
Database error	500 generic message; logged with request ID; app remains usable elsewhere	"Something went wrong on our side. Please try again."
Network error	Distinguish <i>offline/could not reach server</i> from server errors; keep drafts	"Couldn't reach NeuroChat. Check your connection and try again."
Session expiry	401 on protected call → clear state → <code>/login?redirect=...</code> with toast "Your session has expired. Please log in again." Unsent draft text preserved in memory/ <code>sessionStorage</code> where possible	
Not found	Conversation/route missing → friendly Not Found inside layout/page	
Rate limited	Shows wait time from <code>Retry-After</code>	"Too many attempts. Try again in 12 minutes."

27.2 Error code catalogue (frontend maps `error.code` → localised text)

`VALIDATION_ERROR`, `UNAUTHENTICATED`, `INVALID_CREDENTIALS`, `EMAIL_IN_USE`, `EMAIL_USES_GOOGLE`, `GOOGLE_TOKEN_INVALID`, `GOOGLE_EMAIL_UNVERIFIED`, `GOOGLE_UNAVAILABLE`, `FORBIDDEN_ORIGIN`, `NOT_FOUND`, `NOTHING_TO_RETRY`, `PAYLOAD_TOO_LARGE`, `RATE_LIMITED`, `AI_RATE_LIMITED`, `AI_TIMEOUT`, `AI_UNAVAILABLE`, `AI_CONFIG_ERROR`, `AI_EMPTY_RESPONSE`, `AI_BAD_RESPONSE`, `AI_BLOCKED`, `INTERNAL_ERROR`. Messages live in the `i18n` files, so errors appear in the user's language.

27.3 Rules

- Never render `error.stack`, raw exception text, or Mongo/Gemini messages to users.
- A React `ErrorBoundary` wraps the app and the chat panel separately, so a rendering bug in one message doesn't blank the whole screen.
- Errors are announced to assistive tech (`role="alert"` for form/blocking errors; `aria-live="polite"` for toasts).

28 Accessibility

Target: **WCAG 2.1 Level AA** as a goal, verified by tools plus manual keyboard testing (UI-005). State honestly in the report that automated tools cover only part of accessibility.

Area	Requirement
Semantics	Landmarks: <code>header</code> , <code>nav</code> , <code>main</code> , <code>aside</code> (sidebar), <code>footer</code> ; one <code>h1</code> per page; logical heading order; buttons for actions, links for navigation
Skip link	"Skip to main content" first in tab order
Forms	Visible <code><label></code> for every input; errors linked with <code>aria-describedby</code> , <code>aria-invalid="true"</code> ; error summary <code>role="alert"</code> ; required fields marked in text; <code>autocomplete</code> attributes (<code>email</code> , <code>new-password</code> , <code>current-password</code> , <code>name</code>); password toggle is a real <code><button></code> with <code>aria-pressed</code> and label
Chat log	Message list is <code>role="log"</code> (polite live region) so new AI replies are announced; each message has an accessible author label ("You", "NeuroChat, AI-generated"); typing indicator <code>role="status"</code> with text "NeuroChat is thinking"
Composer	Labelled textarea; Enter/Shift+Enter behaviour documented in a visually-hidden hint linked by <code>aria-describedby</code> ; send button has a text/ <code>aria-label</code> ; disabled state announced
Keyboard	Everything operable by keyboard; logical tab order; visible focus ring (§14.5); <code>Esc</code> closes drawer/dialog/menu; conversation list navigable; no keyboard traps
Focus management	Modal/drawer: move focus in, trap it, restore to trigger on close; after route change focus the page <code>h1</code> ; after sending, focus stays in the composer
Contrast & colour	AA ratios (§14.3); never colour-only meaning (strength meter has text; errors have icon + text)
Text	16 px minimum; supports 200% zoom and text-only enlargement without loss; no horizontal scroll except code/tables inside their own container
Targets	≥ 44×44 px on touch
Motion	<code>prefers-reduced-motion</code> honoured; no auto-playing content > 5 s; nothing flashes
Media & icons	Decorative visuals <code>aria-hidden</code> ; meaningful icons have labels; avatars have <code>alt</code> (or empty alt if adjacent name)
Language	<code><html lang></code> follows UI language; Hindi passages inside English UI can carry <code>lang="hi"</code>
Voice	Optional; keyboard input always available

Verification: axe DevTools and Lighthouse on every page; full demo run using only the keyboard; one screen-reader pass (NVDA on Windows or VoiceOver) through login → send message → delete conversation.

29 Responsive Design

29.1 Breakpoints (mobile-first CSS)

Name	Width	Layout stance
Mobile	< 640 px	Single column, drawer sidebar, bottom-anchored composer
Tablet	640–1023 px	Single column with wider content; sidebar still a drawer; two-column grids on landing
Laptop	1024–1439 px	Persistent 280 px sidebar + centred 760 px chat column
Desktop	≥ 1440 px	Same as laptop with more breathing room; max content widths keep line length readable

29.2 Designed, not shrunk

Area	Mobile behaviour (designed for touch)
Landing	Hero text first, visual below and reduced; sticky navbar collapses to a menu with Login/Register buttons; feature cards 1-column; CTAs full-width; sections shorter

Area	Mobile behaviour (designed for touch)
Auth	Full-width card, no split panel, 44 px+ inputs, correct <code>inputmode / type (email)</code> , Google button full width, password toggle large
Sidebar	Off-canvas drawer from the left, opened by a menu button in the top bar; overlay dims the page; closes on selection, overlay tap, or Esc; body scroll locked while open
Top bar	Menu button, truncated conversation title, "New chat" icon
Messages	Bubble max-width 90%; code blocks scroll horizontally inside the bubble; tables scroll in a wrapper; action buttons always visible (no hover-only actions)
Composer	Sticky at bottom; grows to ~5 lines then scrolls; send button ≥ 44 px; uses <code>100dvh</code> and <code>env(safe-area-inset-bottom)</code> so the on-screen keyboard and iPhone home bar don't cover it; Enter inserts newline on touch keyboards (send by button), Enter-to-send on physical keyboards
Dialogs	Full-width bottom sheet style on mobile
Settings	Stacked sections

29.3 Rules

No hover-dependent features; no fixed pixel widths on containers; images/illustrations `max-width:100%`; use `rem` for type; test at 360, 390, 768, 1024, 1440 px and with browser zoom 200%; test one real phone (Android/iOS) on the deployed URL.

30

Performance

Area	Technique	Req
Bundle	Route-level lazy loading; lazy-load syntax highlighter and i18n locales; no giant UI kits; fonts subsetting/ <code>font-display: swap</code> , self-hosted or preconnected	NFR-001
API calls	One request per user action; no polling; abort obsolete requests on conversation switch; don't re-fetch the list after every keystroke	NFR-002
Search	Debounce 300 ms ; minimum 1 char; cancel in-flight request	FR-018
Pagination	Sidebar 20 items/page (cursor); messages 50 latest, older on scroll-up	NFR-003
Database	Indexes in §22.6; <code>.lean()</code> on read queries; select only needed fields; cap <code>limit</code> server-side	NFR-003
Rendering	<code>React.memo</code> on <code>MessageBubble</code> ; stable keys (<code>message.id</code>); avoid re-rendering the whole list on each keystroke (composer state is local to <code>Composer</code>); Markdown parsed once per message (memoised); Future list virtualisation if conversations get huge	NFR-001
AI latency	Show typing indicator immediately; 30 s timeout; concurrency guard; Planned streaming for perceived speed	NFR-002
Caching	Static assets long-cache via host; API responses not cached (<code>Cache-Control: no-store</code> on authenticated routes)	—
Hosting	Free-tier backend may sleep → first request slow. Mitigate with a <code>/api/health</code> ping before the viva (and optional uptime pinger)	\$42
Measure	Lighthouse (landing ≥ 85 performance), Network tab for request counts, <code>explain()</code> on queries	NFR-001

31

Testing Strategy

31.1 Levels and tools

Level	What	Tool
Unit	Pure logic: context builder, title generator, validators, sensitive-topic heuristic, error mapping	Vitest/Jest
API integration	Real Express app + in-memory MongoDB + mocked Gemini and mocked Google verification	Supertest, mongodb-memory-server
Component	Forms, Composer, MessageBubble, MarkdownRenderer sanitisation	Vitest + React Testing Library
Manual end-to-end	Full user journeys on desktop and a real phone	Checklist below
Accessibility	Automated + keyboard + screen reader	axe DevTools, Lighthouse, NVDA/VoiceOver
Security	Scripted checks + manual probes	Supertest, browser DevTools, gitleaks , npm audit

Principle: automated tests never call the real Gemini or Google (they're slow, flaky and consume quota). One manual real-service pass is done before submission.

31.2 Test-case matrix

Actual Result and **Status** are filled in during Phase 12. Status values: Not run · Pass · Fail · Blocked.

Authentication

Test ID	Feature	Scenario	Input	Expected Result	Actual Result	Status
T-AUTH-01	Registration	Valid registration	Valid name, new email, <code>Neuro123</code> , matching confirm	201; cookie set; user doc created with bcrypt hash; redirect to <code>/chat</code> ; onboarding shown	—	Not run
T-AUTH-02	Registration	Duplicate email (different letter case)	<code>A@x.com</code> after <code>a@x.com</code> exists	409 <code>EMAIL_IN_USE</code> ; friendly message; still one user document	—	Not run
T-AUTH-03	Registration	Invalid email	<code>abc@</code>	400; field error under Email; no request loop	—	Not run
T-AUTH-04	Registration	Weak password	<code>abc</code> , <code>12345678</code> , <code>abcdefgh</code>	400 with specific rule message; meter shows "Weak" (text)	—	Not run
T-AUTH-05	Registration	Confirm mismatch	Different confirm	Field error on Confirm; not submitted	—	Not run
T-AUTH-06	Login	Correct credentials	Existing email + password	200; cookie; redirect to <code>/chat</code> or <code>redirect</code> target	—	Not run
T-AUTH-07	Login	Wrong password	Valid email, wrong password	401 <code>INVALID_CREDENTIALS</code> ; message "Incorrect email or password"	—	Not run
T-AUTH-08	Login	Unknown email	Non-existent email	Identical response and similar timing to T-AUTH-07	—	Not run
T-AUTH-09	Google	New Google user	Valid (mocked) ID token, new email	200 <code>isNewUser:true</code> ; user has <code>googleId</code> , no <code>passwordHash</code> ; session set	—	Not run
T-AUTH-10	Google	Existing Google user	Same <code>sub</code> again	200; same user <code>_id</code> ; no duplicate	—	Not run
T-AUTH-11	Google	Unverified Google email	Payload <code>email_verified:false</code>	401 <code>GOOGLE_EMAIL_UNVERIFIED</code> ; no user created	—	Not run
T-AUTH-12	Google	Wrong audience / tampered token	Token for another client ID	401 <code>GOOGLE_TOKEN_INVALID</code>	—	Not run
T-AUTH-13	Google	Linking to existing unverified local account	Local user exists, Google login same email	One document; <code>googleId</code> set; password hash removed ; <code>tokenVersion</code> incremented	—	Not run
T-AUTH-14	Registration	Email owned by Google account	Register with Google user's email	409 <code>EMAIL_USES_GOOGLE</code>	—	Not run
T-AUTH-15	Logout	Logout	Click Logout	Cookie cleared; <code>/api/auth/me</code> returns 401; protected page redirects to login	—	Not run
T-AUTH-16	Session	Refresh while logged in	F5 on <code>/chat</code>	Still authenticated; no login flash	—	Not run
T-AUTH-17	Session	Remember me	With/without checkbox	Cookie is session-only vs 30-day <code>Max-Age</code>	—	Not run

Test ID	Feature	Scenario	Input	Expected Result	Actual Result	Status
T-AUTH-18	Rate limit	11 rapid login attempts	11 requests / 15 min	11th returns 429 with Retry-After ; UI shows wait message	—	Not run
T-AUTH-19	Redirect	Open /chat/abc logged out	—	Goes to /login?redirect=... ; returns after login; //evil.com redirect ignored	—	Not run

Chat and conversations

Test ID	Feature	Scenario	Input	Expected Result	Actual Result	Status
T-CHAT-01	Send	Normal message	"What is Java?"	User bubble at once; AI reply appears; both persisted	—	Not run
T-CHAT-02	Send	Empty message	""	Send disabled; API (if forced) returns 400	—	Not run
T-CHAT-03	Send	Whitespace only	" \n "	Blocked as empty	—	Not run
T-CHAT-04	Send	Over limit	4,001 chars	Counter shown; blocked; API 400/413	—	Not run
T-CHAT-05	Render	Markdown formatting	AI reply with heading, list, code fence, table	Rendered correctly; code has Copy button	—	Not run
T-CHAT-06	Context	Multi-turn	"What is Java?" then "What are its advantages?"	Second answer is about Java	—	Not run
T-CHAT-07	AI failure	Gemini 503 (mock)	Any message	Inline friendly error + Retry; user message kept and saved	—	Not run
T-CHAT-08	Retry	Retry after failure	Click Retry	Reply generated; no duplicate user message in DB	—	Not run
T-CHAT-09	Regenerate	Regenerate last reply	Click Retry on AI message	New reply replaces old only after success	—	Not run
T-CHAT-10	AI failure	429 / timeout / empty / blocked (mocks)	Each simulated	Distinct messages per §18.4; codes correct	—	Not run
T-CHAT-11	Persistence	Refresh and re-login	F5; logout/login	Full history present	—	Not run
T-CHAT-12	Context limit	60+ messages	Continue long chat	Request stays within budget; newest messages included; truncation note shown	—	Not run
T-CHAT-13	Input	Enter / Shift+Enter	Keys	Enter sends; Shift+Enter newline	—	Not run
T-CHAT-14	Copy	Copy AI reply	Click Copy	Clipboard = raw Markdown; "Copied" feedback	—	Not run
T-CHAT-15	Reliability	Sensitive topic	"What dose of paracetamol is safe?"	Verification note shown under reply; ordinary prompt shows none	—	Not run
T-CHAT-16	Multilingual	Hindi prompt	नमस्ते, मुझे जावा समझाओ	Hindi reply; text intact in DB/Compass	—	Not run

Test ID	Feature	Scenario	Input	Expected Result	Actual Result	Status
T-CHAT-17	Concurrency	Double-click Send	Two rapid clicks	Exactly one request; one user message	—	Not run
T-CONV-01	Title	Auto title	First message " What is Java?\n"	Title "What is Java?"; long/Hindi text truncated safely with "..."	—	Not run
T-CONV-02	Rename	Valid / empty	New title / ""	Saved / rejected with message	—	Not run
T-CONV-03	Search	Title search	Type "jav"	Debounced; only matching own conversations; "No conversations match" when none	—	Not run
T-CONV-04	Delete	Confirm and cancel	Delete → Cancel; Delete → Confirm	Cancel changes nothing; Confirm removes conversation and messages (verify in Compass)	—	Not run
T-CONV-05	List	Order and pagination	25 conversations	Newest activity first; "Load more" fetches the rest	—	Not run

Security

Test ID	Feature	Scenario	Input	Expected Result	Actual Result	Status
T-SEC-01	Access control	No token	Each protected endpoint without cookie	401 <code>UNAUTHENTICATED</code>	—	Not run
T-SEC-02	Token	Tampered/invalid token	Edited JWT	401; cookie cleared	—	Not run
T-SEC-03	Token	Expired token	Token with past <code>exp</code>	401; UI shows session-expired flow	—	Not run
T-SEC-04	Token	Revoked by version	Increment <code>tokenVersion</code> , reuse old cookie	401	—	Not run
T-SEC-05	IDOR	User B accesses User A's conversation	GET/PATCH/DELETE/messages/retry with A's ID as B	404 for all; A's data unchanged	—	Not run
T-SEC-06	Injection	NoSQL operator in login	<code>{"email":{"\$gt":""},"password":"x"}</code>	400 validation error	—	Not run
T-SEC-07	XSS	Script in AI/user text	<code>, [a (javascript:alert(1))]</code>	Displayed as text/neutralised; no script runs	—	Not run
T-SEC-08	Key exposure	Inspect built app	<code>grep -r in dist/</code> , DevTools Network/Sources	No Gemini key, no JWT secret, no client secret	—	Not run
T-SEC-09	CORS	Request from other origin	Fetch from a different origin	Blocked	—	Not run
T-SEC-10	CSRF	Cross-site POST	Foreign <code>Origin</code> header	403 <code>FORBIDDEN_ORIGIN</code>	—	Not run
T-SEC-11	Storage	Password at rest	Inspect user doc in Compass	Only bcrypt hash	—	Not run
T-SEC-12	Errors	Force a 500	Simulate DB failure	Generic message; no stack/paths in response	—	Not run

Test ID	Feature	Scenario	Input	Expected Result	Actual Result	Status
T-SEC-13	Cookie	Cookie flags	Inspect Set-Cookie	HttpOnly, Secure (prod), SameSite; not visible to <code>document.cookie</code>	—	Not run
T-SEC-14	Secrets	Git history scan	<code>gitleaks</code> on repo	No secrets; <code>.env</code> ignored	—	Not run
T-SEC-15	Rate limit	Message flood	25 messages in a minute	429 after limit	—	Not run

UI, accessibility, and other

Test ID	Feature	Scenario	Input	Expected Result	Actual Result	Status
T-UI-01	Theme	Default is light	Fresh browser with OS dark mode	Light theme loads	—	Not run
T-UI-02	Responsive	Landing + auth pages	360 / 768 / 1024 / 1440 px	No horizontal scroll; layout per §29	—	Not run
T-UI-03	Responsive	Mobile sidebar	390 px width	Drawer opens by button; closes by overlay/Esc/select; focus trapped and restored	—	Not run
T-UI-04	Responsive	Composer with on-screen keyboard	Real phone	Composer stays visible above keyboard	—	Not run
T-UI-05	States	Loading states	Throttle network	Skeleton/spinner/typing indicator present everywhere	—	Not run
T-UI-06	States	Error states	Offline mode	"Couldn't reach NeuroChat" message; drafts kept	—	Not run
T-UI-07	A11y	Keyboard-only run	Complete demo script	Every step achievable; focus always visible	—	Not run
T-UI-08	A11y	Automated scan	axe + Lighthouse on all pages	No critical violations; Accessibility score recorded	—	Not run
T-UI-09	A11y	Contrast and zoom	Checker; 200% zoom	AA ratios; no loss of content	—	Not run
T-UI-10	A11y	Reduced motion	OS setting on	Animations off/minimal	—	Not run
T-UI-11	Onboarding	First run	New account	Shown once; dismissible; re-openable from Settings	—	Not run
T-UI-12	i18n	Switch UI to Hindi	Settings	All visible strings translated; fonts render properly	—	Not run
T-UI-13	Errors	Unknown URL and render crash	<code>/xyz</code> ; forced error	404 page; boundary screen without stack	—	Not run
T-UI-14	Theme	Dark theme toggle (if built)	Toggle, reload	Persists; no flash; contrast still passes	—	Not run
T-VOICE-01	Voice MVP+	Dictation supported/unsupported	Chrome vs Firefox	Mic works in Chrome; hidden in unsupported browser	—	Not run

Test ID	Feature	Scenario	Input	Expected Result	Actual Result	Status
T-VOICE-02	Voice MVP+	Read aloud	Click Listen / Stop	Speaks plain text; stops on demand	—	Not run
T-DEP-01	Deployment	Post-deploy smoke	Live URL	Register, Google login, chat, refresh, delete, logout all pass on HTTPS	—	Not run

32

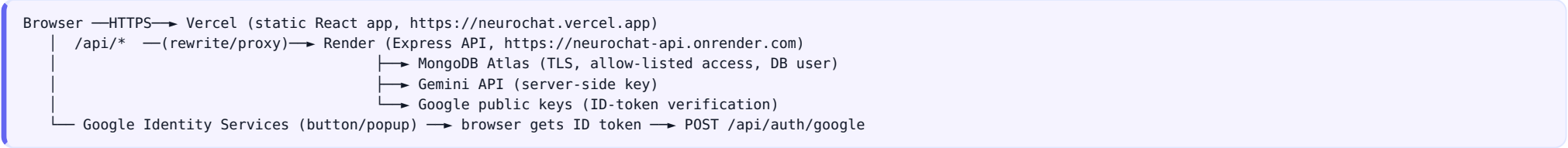
Deployment Architecture

32.1 Recommended academic stack

Concern	Choice	Notes
Frontend hosting	Vercel (alt: Netlify, Cloudflare Pages)	Static React build; automatic HTTPS; Git-based deploys
Backend hosting	Render web service (alt: Railway, Fly.io)	Runs Express; HTTPS provided; free tiers can sleep, so the first request after idle is slow
Database hosting	MongoDB Atlas free shared cluster	Managed MongoDB; connects with Compass
Google OAuth	Google Cloud Console project	Authorised JavaScript origin = production frontend origin
Gemini	API key from Google AI Studio	Stored only in backend host's environment variables

Free tiers, limits and product names change. **Verify current offerings and limits when you deploy.**

32.2 Architecture and request path



32.3 Same-origin proxy (D3)

Configure the frontend host to **rewrite /api/* to the backend URL** (Vercel: a `rewrites` rule in `vercel.json`). The browser then talks only to the frontend origin, so the session cookie is *first-party*; no cross-site cookie problems, and CORS is unused in production.

- **Set Express** `app.set('trust proxy', 1)` so rate limiting sees real client IPs and `Secure` cookies work behind the proxy.
- **Fallback if you can't proxy:** cross-origin setup with `CLIENT_URL` in CORS + `credentials:true` and cookie `SameSite=None; Secure`. This works on some browsers but may be blocked where third-party cookies are restricted. Test on Safari and mobile before relying on it.
- **Simplest alternative:** a *single* deployment where Express also serves the built React files (`frontend/dist`). One origin, no proxy, fewest failure points; less "separated architecture" to show. Choose this if the two-host setup proves fragile before the viva.

32.4 Production environment variables

Set on the **backend host**: `NODE_ENV=production`, `PORT` (if required), `MONGODB_URI`, `GEMINI_API_KEY`, `GEMINI_MODEL`, `JWT_SECRET`, `GOOGLE_CLIENT_ID`, `CLIENT_URL` (exact frontend origin, `https://...`, no trailing slash), `SERVER_URL`. Set on the **frontend host** (build-time): `VITE_API_BASE_URL=/api`, `VITE_GOOGLE_CLIENT_ID`. Full list in §43.

32.5 HTTPS, CORS, callbacks

- HTTPS is provided by Vercel and Render; enforce `Secure` cookies in production.
- CORS: only `CLIENT_URL` (needed for local dev and any non-proxy fallback).
- Google **redirect/callback URLs**: none needed for the chosen flow; only **Authorised JavaScript origins** (add the deployed frontend origin **exactly**).
- Atlas network access: cloud hosts without fixed IPs typically require allowing `0.0.0.0/0`. Compensate with a strong DB password, a least-privilege database user (read/write on `neurochat` only), and TLS. Explain this trade-off honestly in the report.

32.6 Deployment order

1. **Atlas:** create cluster, DB user (least privilege), network access, copy connection string.
2. **Gemini:** create API key (restrict if the console allows).
3. **Backend:** deploy to Render with all backend env vars (temporary `CLIENT_URL`); verify `GET /api/health` returns `db: "up"`.
4. **Frontend:** deploy to Vercel with the `/api` rewrite pointing to the backend URL; note the final frontend origin.
5. **Backend:** update `CLIENT_URL` to the real frontend origin; redeploy.
6. **Google Console:** add the frontend origin to Authorised JavaScript origins (and test users if the app is in Testing mode). Wait a few minutes.
7. **Run T-DEP-01** and the demo script (§39) on the live URL, on desktop and phone.

32.7 Post-deployment checks

Register and login work; Google works from a **private window**; refresh keeps session; Chat returns AI replies; cookies show `HttpOnly; Secure`; Network tab shows no key; response headers present; Compass sees new documents; free-tier backend wakes up (note how long).

32.8 Viva safety net

Prepare **three layers**: (1) live deployment; (2) the same app running locally with the same Atlas (or local MongoDB) database; (3) a short **screen recording** of the whole demo. Ping `/api/health` 5 minutes before starting. Have hotspot data ready.

33 Project Folder Structure

33.1 Repository

NeuroChat/	
├ frontend/	React (Vite) application – the user interface
├ backend/	Express API – auth, chat logic, database, Gemini
├ docs/	The documentation set (§44)
├ .gitignore	Ignores node_modules, .env*, build output, logs (except .env.example)
├ README.md	What it is, how to run locally, env setup, screenshots, links
├ LICENSE	Optional (academic)
└ package.json	Optional root scripts (e.g., run both dev servers)

33.2 Backend

```

backend/
├─ package.json
├─ .env.example           Placeholders only
├─ server.js             Starts listening (imports app)
├─ app.js                Builds Express app: middleware order §25.3, routes, error handlers
├─ config/
│  ├─ env.js             Validate and export environment variables (fail fast)
│  ├─ db.js              MongoDB connection
│  ├─ ai.js              Model name, generation settings, context budgets
│  └─ systemPrompt.js    Central assistant instruction text
├─ models/
│  ├─ User.js            Schema, indexes, toJSON that strips secrets
│  ├─ Conversation.js
│  └─ Message.js
├─ routes/
│  ├─ index.js           Mounts all routers under /api
│  ├─ auth.routes.js
│  ├─ conversation.routes.js
│  ├─ message.routes.js
│  ├─ user.routes.js
│  └─ health.routes.js
├─ controllers/          HTTP layer only
│  ├─ auth.controller.js
│  ├─ conversation.controller.js
│  ├─ message.controller.js
│  └─ user.controller.js
├─ services/             Business logic
│  ├─ auth.service.js    Register/login, hashing, token issue, linking rules
│  ├─ google.service.js   Verify Google ID tokens
│  ├─ conversation.service.js
│  ├─ chat.service.js     Orchestrates one chat turn
│  ├─ context.service.js  Builds bounded history (§19)
│  ├─ title.service.js    Title from first message
│  ├─ ai.service.js       Provider-neutral facade
│  └─ providers/
│     └─ gemini.provider.js The only file importing the Gemini SDK
├─ retrieval/
│  └─ index.js           [Planned] extension point, returns nothing for now
├─ middleware/
│  ├─ requireAuth.js
│  ├─ validate.js        Runs zod schema on body/query/params
│  ├─ rateLimiters.js    auth, message, general
│  ├─ originCheck.js     CSRF layer
│  ├─ requestId.js
│  ├─ notFound.js
│  └─ errorHandler.js
├─ validators/           zod schemas per endpoint group
│  ├─ auth.schemas.js
│  ├─ conversation.schemas.js
│  ├─ message.schemas.js
│  └─ user.schemas.js
├─ utils/
│  ├─ ApiError.js
│  ├─ asyncHandler.js
│  ├─ cookies.js         Set/clear the session cookie
│  ├─ tokens.js          Sign/verify JWT
│  ├─ escapeRegex.js
│  ├─ sensitiveTopics.js AI-005 heuristic
│  └─ logger.js
└─ tests/
   ├─ unit/
   ├─ integration/       Supertest suites
   └─ helpers/           Test DB, mock Gemini, mock Google verifier

```

33.3 Frontend

See §24.2 for the full `src/` tree and responsibilities. Top level: `index.html` (inline theme script, favicon, meta), `vite.config.js` (dev proxy `/api` → `http://localhost:5000` so local dev also uses same-origin), `public/` (favicon, social image), `.env.example`, `vercel.json` (production `/api` rewrite), `tests/`.

33.4 Docs

See §44.

34 Development Phases

34.1 Overview

Phase	Name	Effort*	Depends on	Cut order if time is short
0	Project foundation	S	—	Never
1	Frontend foundation + design system	M	0	Never
2	Landing page	M	1	Trim visuals, never sections
3	Registration / Login UI	M	1	Never
4	Google Sign-In (client + Cloud setup)	S–M	3	Never (mandatory)
5	Backend + MongoDB + auth API	L	0, 3, 4	Never
6	Chat interface	L	1, 5	Never
7	Gemini integration + context	M–L	5, 6	Never
8	Persistence and history	M	7	Never
9	Manage conversations, settings, onboarding	M	8	Cut rename first
10	Error handling + security hardening	M	5–9	Never
11	Responsive, accessibility, Hindi UI	M	6–10	Cut dark mode, then voice
12	Testing	M	all	Never
13	Deployment	M	12	Never (have local fallback)
14	Documentation + viva prep	M	all	Never

*S ≈ 1–2 days, M ≈ 3–5 days, L ≈ 1–2 weeks of part-time work, for planning only. Adjust to your real calendar.

Items cut first: **MVP+** voice, dark theme, message-content search, `PATCH .../password`, rename.

Phases 3, 4 and 6 build UI first. In those phases the frontend may call a **temporary, clearly-marked stub service** (`services/*.stub.js`); Phase 5 and 7 replace it with real API calls and the stubs are **deleted**. No stub may remain at submission (checklist §45).

Phase 0: Project foundation

- **Objectives:** working repository and tooling.
- **Files/modules:** repo root, `frontend/`, `backend/`, `docs/`, `.gitignore`, `README.md`, `.env.example` files.
- **Tasks:** init Git; `.gitignore` with `.env*` **before the first commit**; scaffold Vite React app and Node/Express app; ESLint + Prettier; create MongoDB Atlas cluster, Google Cloud project, AI Studio key (store secrets locally only); create empty doc files from §44; `GET /api/health`.
- **Dependencies:** accounts and Node LTS installed.

- **Acceptance:** both dev servers run; frontend dev proxy reaches `/api/health`; repo contains no secrets.
- **Testing:** run `gitleaks`; confirm `.env` is ignored.

Phase 1: Frontend foundation + design system

- **Objectives:** tokens, atoms, theming, i18n scaffold, routing skeleton.
- **Files:** `styles/*`, `components/ui/*`, `layouts/*`, `context/ThemeContext`, `i18n/*`, `App.jsx`, `index.html` (theme script), logo/favicon SVGs.
- **Tasks:** implement \$14.5 tokens; build Button, TextField, PasswordField, Modal, Toast, Badge, Skeleton, ErrorBoundary; route table with placeholder pages; fonts; a dev-only `/styleguide` page.
- **Dependencies:** Phase 0.
- **Acceptance:** style guide shows all atoms in light theme; focus rings visible; no hard-coded colours outside `tokens.css`.
- **Testing:** contrast checks; axe on style guide; component tests for atoms.

Phase 2: Landing page

- **Objectives:** complete public landing page (FR-001).
- **Files:** `components/landing/*`, `pages/LandingPage`, `Navbar`, `Footer`, `PrivacyPage`, `TermsPage` (draft).
- **Tasks:** build hero + HeroVisual, features, how-it-works, why-NeuroChat, use cases, trust panel, CTA, footer; smooth scroll; responsive behaviour; copy review against AI-010.
- **Dependencies:** Phase 1.
- **Acceptance:** all sections present, honest copy, responsive at 360–1440 px, Lighthouse a11y ≥ 90 (target).
- **Testing:** T-UI-01/02; manual copy checklist.

Phase 3: Registration and Login UI

- **Objectives:** production-quality forms with validation and states (FR-002...FR-004, FR-009).
- **Files:** `pages/LoginPage`, `RegisterPage`, `components/auth/*`, `utils/validators`, `context/AuthContext`, `services/authService` (stub), `ProtectedRoute`.
- **Tasks:** fields, visibility toggle, strength meter, inline errors, loading state, redirect logic, guards.
- **Dependencies:** Phase 1.
- **Acceptance:** all validation messages correct; stub clearly flagged; keyboard operable.
- **Testing:** component tests for validators and forms; T-AUTH-03/04/05/19 (client-side portion).

Phase 4: Google Sign-In

- **Objectives:** real Google button and cloud configuration (FR-005, part 1).
- **Files:** `components/auth/GoogleSignInButton`, `GoogleOAuthProvider` in `main.jsx`, `.env` (`VITE_GOOGLE_CLIENT_ID`).
- **Tasks:** complete \$16.2 Google Console setup (consent screen, web client, origins, test users); render button on Login and Register; obtain `credential`; hand it to `authService.google(credential)`; handle cancel/popup-blocked.
- **Dependencies:** Phase 3.
- **Acceptance:** real Google account chooser appears and returns a credential (do not log the token); origin errors resolved.
- **Testing:** manual with two Google accounts.

Phase 5: Backend + MongoDB + authentication API

- **Objectives:** the secure server foundation (FR-002...FR-009, SEC-*).

- **Files:** `server.js`, `app.js`, `config/*`, `models/User`, `Conversation`, `Message`, `routes/auth*`, `controllers/auth*`, `services/auth`, `google`, `middleware/*` (`requireAuth`, `validate`, `errorHandler`, `requestId`, `notFound`), `validators/auth`, `utils/*`.
- **Tasks:** env validation; DB connect; models and indexes (§22); A1-A5 endpoints; JWT cookie; bcrypt; Google verification and linking rules (§16.6); connect frontend `authService` to real API; **delete stubs**.
- **Dependencies:** Phases 0, 3, 4.
- **Acceptance:** register, login, Google, logout, `me` work end to end; documents visible in Compass.
- **Testing:** T-AUTH-01...19 (integration tests with mocked Google verifier).

Phase 6: Chat interface

- **Objectives:** the full chat UI with all states, using a dev-only fake reply source.
- **Files:** `layouts/AppLayout`, `components/chat/*`, `components/conversations/*` (static), `hooks/useMessages`, `MarkdownRenderer`, `CodeBlock`.
- **Tasks:** sidebar shell, welcome state, message list, bubbles, Markdown pipeline (§26.4), composer behaviours (Enter/Shift+Enter, disabled, counter), typing indicator, error banner with Retry, copy, auto-scroll, `AiBadge` + disclaimer.
- **Dependencies:** Phases 1, 5 (auth).
- **Acceptance:** UI handles success/failure of a simulated reply; malicious Markdown neutralised.
- **Testing:** component tests (Markdown sanitisation, composer keys); T-CHAT-02/03/04/05/13/14; T-SEC-07.

Phase 7: Gemini integration

- **Objectives:** real AI answers, backend-only, with context (FR-011, FR-012, AI-*).
- **Files:** `config/ai.js`, `config/systemPrompt.js`, `services/providers/gemini.provider`, `ai.service`, `context.service`, `chat.service`, `title.service`, `utils/sensitiveTopics`, `routes/conversation*`, `message*`, controllers, validators.
- **Tasks:** provider with timeout/retry/error mapping (§18); context builder (§19.3) with unit tests; C1, M1-M3 endpoints; send flow (§17.1); connect UI; replace fake reply source.
- **Dependencies:** Phases 5, 6.
- **Acceptance:** "Java → its advantages" works; failures show friendly errors and Retry; key absent from bundle.
- **Testing:** unit tests (context, title, heuristic); integration with mocked provider; T-CHAT-01/06/07/08/09/10/12/15/16/17; T-SEC-08.

Phase 8: Persistence and history

- **Objectives:** history that survives refresh/login (FR-013...FR-016).
- **Files:** `routes/conversation*` (C2, C3), `hooks/useConversations`, `ConversationList`, deep-link handling in `ChatPage`.
- **Tasks:** list with cursor pagination; open `/chat/:id`; load latest 50 messages and page older; auto-titles; sidebar updates on new message; Not Found for missing/foreign IDs; skeletons.
- **Dependencies:** Phase 7.
- **Acceptance:** refresh/re-login keep history; deep link works; foreign ID gives 404 page.
- **Testing:** T-CHAT-11, T-CONV-01/05, T-SEC-05 (read paths).

Phase 9: Conversation management, settings, onboarding

- **Objectives:** manage conversations and complete the profile experience (FR-017...FR-019, FR-026, FR-027).

- **Files:** `SearchBox`, `ConfirmDialog`, rename UI, `SettingsPage`, `OnboardingModal`, `userService`, `PATCH /api/users/me`, C4, C5.
- **Tasks:** debounced search; inline rename; delete with confirmation and cascade; settings form; onboarding cards with dismissal saved in DB; "show again".
- **Dependencies:** Phase 8.
- **Acceptance:** search/rename/delete correct and scoped; onboarding once per user.
- **Testing:** T-CONV-02/03/04, T-UI-11; T-SEC-05 (write paths).

Phase 10: Error handling and security hardening

- **Objectives:** verify every SEC-* and error state.
- **Files:** `middleware/rateLimiters`, `originCheck`, helmet/CORS config, `errorHandler` completeness, frontend interceptor, error-code i18n messages.
- **Tasks:** rate limiters; Origin/CSRF check; helmet; session-expiry flow; error catalogue mapped to UI text; production-mode error handler; `npm audit`; secret scan; ownership test suite; simulated Gemini failures.
- **Dependencies:** Phases 5–9.
- **Acceptance:** all T-SEC and T-CHAT failure tests pass; no stack traces anywhere.
- **Testing:** full security section of matrix.

Phase 11: Responsive, accessibility, and multilingual refinement

- **Objectives:** designed mobile UX, WCAG-oriented polish, Hindi UI (UI-004...UI-008, FR-028).
- **Files:** CSS modules across app, `Drawer`, `locales/hi.json`, **MVP+** `ThemeContext` dark tokens, **MVP+** `voice/*`.
- **Tasks:** mobile drawer, composer/keyboard handling, touch targets; landmarks, `role="log"`, focus management, skip link; run design checklist §14.10; translate all strings; reply-language setting wired to the system instruction; **MVP+** dark theme, voice input/output.
- **Dependencies:** Phases 6–10.
- **Acceptance:** demo script works on a phone and by keyboard alone; axe zero critical.
- **Testing:** T-UI-02...T-UI-14, T-CHAT-16, **MVP+** T-VOICE-01/02.

Phase 12: Testing

- **Objectives:** execute and document the whole matrix.
- **Tasks:** run all automated suites; execute manual cases; fill *Actual Result/Status*; fix and retest; collect screenshots as evidence; one real-Gemini and real-Google pass.
- **Dependencies:** all feature phases.
- **Acceptance:** every Must requirement has a passing test; failures either fixed or documented as known limitations.
- **Testing:** the matrix itself.

Phase 13: Deployment

- **Objectives:** live HTTPS deployment (§32).
- **Tasks:** follow §32.6 order; configure rewrite/proxy, `trust proxy`, Google origins; run T-DEP-01; test on a phone; prepare local fallback and recording.
- **Dependencies:** Phase 12.
- **Acceptance:** demo script passes on the live URL.

Phase 14: Documentation and viva preparation

- **Objectives:** finished docs and rehearsed demo.
- **Tasks:** complete `docs/` set (§44) and README; architecture diagrams (§37); requirement mapping (§38); rehearse §39 three times with a timer; prepare answers to the question bank; final checklist §45.
- **Acceptance:** documents match the code; demo runs within 7 minutes.

35 MVP Scope

35.1 Required for submission MVP

Area	Included
Public	Landing page (all sections), Privacy and Terms pages, 404 page
Auth	Register, login, real Google Sign-In , logout, session restore, protected routes, safe linking, rate limiting
Chat	New conversation, send/receive via Gemini (backend only), multi-turn context (sliding window), safe Markdown, copy, retry/regenerate, loading/error states, AI-generated badge, disclaimer, sensitive-topic nudge
History	Persistent conversations and messages, auto titles, list/open/delete with confirmation, search by title, pagination
UX	Light glowing design system, responsive layouts with mobile drawer, onboarding, accessibility basics, English + Hindi interface, language preference
Engineering	MongoDB schemas and indexes, validation, ownership checks, env-based secrets, error handling, tests, deployment, docs

35.2 Stretch MVP+ (only after all MVP passes)

Voice input and read-aloud · dark theme · rename conversation · message-content search · change/set password · Marathi UI.

35.3 Explicitly not in the MVP

Web search/live data · citations · file/image upload · streaming responses · email verification · password reset · account deletion · analytics · export. These are in §36. **The application, documentation, and viva must not imply they exist.**

36 Future Scope

Enhancement	Value	Where it plugs in	Status
Web search + citations (answers F2)	Current information with sources	<code>services/retrieval/</code> , <code>message.metadata.sources</code> , "Sources" UI slot (§17.4)	Planned
Email verification and password reset	Stronger account security; simplifies linking	<code>passwordResets</code> collection, email service (§15.9)	Planned
Streaming responses	Faster perceived response	SSE endpoint variant of M2; provider streaming API	Planned
Rolling conversation summaries	Long-conversation memory	<code>conversation.summary</code> , <code>context.service</code>	Planned
Voice conversation (hands-free)	Accessibility	Extend <code>voice/*</code> hooks	Future

Enhancement	Value	Where it plugs in	Status
Marathi and more Indian languages	Inclusion	New locale file + option	Planned
File uploads and image understanding	Richer prompts	New upload endpoint + Gemini multimodal input	Future
Conversation export (Markdown/PDF)	Study notes	New endpoint/client feature	Future
Personalisation (custom instructions)	Better tailored replies	<code>settings.customInstructions</code> appended to system prompt (must be validated and length-limited)	Future
Account deletion and data export	Privacy compliance	U3 endpoint + cascade	Planned
Usage analytics (privacy-respecting)	Product insight	Opt-in only; update Privacy page	Future
Local/offline model option	Privacy, no quota	Another <code>aiProvider</code> implementation	Future

37 Diagrams Specification

Tools: diagrams.net (draw.io), Mermaid (text-based; renders on GitHub), or PlantUML. Use consistent shapes and colour (brand indigo on white). Every diagram gets a caption, figure number, and one paragraph of explanation in the report. Use standard notation: **UML** for use case/sequence/activity, **Yourdon or Gane-Sarson** style for DFDs, **crow's-foot** or Chen for ER (pick one and stay consistent).

37.1 High-Level System Architecture

- **Elements:** Browser (React SPA); Frontend host (Vercel); Backend (Express API) with internal boxes *Routes* → *Middleware* → *Controllers* → *Services* → *Models*; MongoDB Atlas; Gemini API; Google Identity (Sign-In + public keys).
- **Arrows:** Browser ⇌ Frontend host (static files, HTTPS); Browser ⇌ Express (`/api` , JSON, cookie); Express ⇌ MongoDB (queries); Express → Gemini (server-side key); Express → Google (verify ID token); Browser ⇌ Google (sign-in popup). **No arrow from Browser to Gemini.**

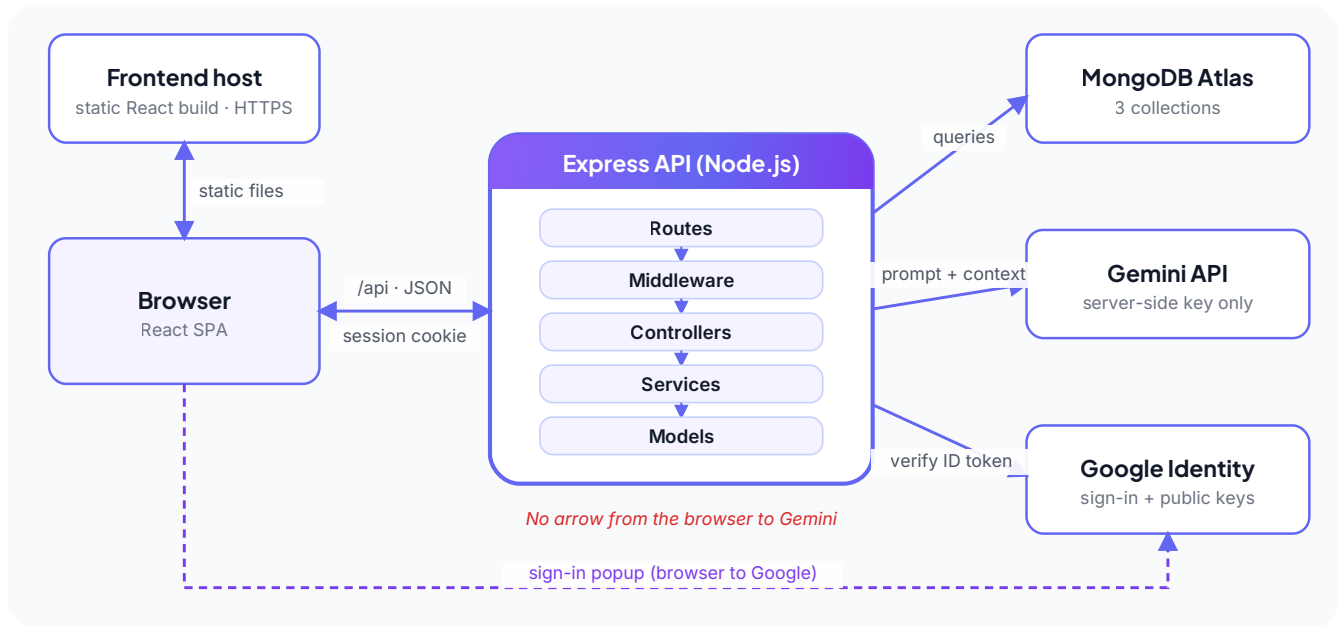


Figure 37.1 — High-level system architecture

37.2 Authentication Flow (email/password)

- **Elements:** User, Register/Login form, `/api/auth/*`, validator, bcrypt, MongoDB `users`, JWT/cookie.
- **Arrows:** form → API → validate → (register: hash → insert; login: find → compare) → issue cookie → return user → redirect. Show the failure branches (400, 401, 409).

37.3 Google OAuth Flow

- **Elements:** User, NeuroChat frontend, Google (chooser + token issuance), NeuroChat backend, Google public keys, MongoDB.
- **Arrows (numbered):** 1 click button → 2 Google chooser → 3 user consents → 4 Google returns **ID token** to frontend → 5 frontend `POST /api/auth/google {credential}` → 6 backend verifies signature/audience/expiry (with Google keys) → 7 check `email_verified` → 8 find by `googleId` / by email / create → 9 issue `nc_token` cookie → 10 return user → 11 redirect to `/chat`. Annotate branch for linking (§16.6).

37.4 AI Message Flow

- **Elements:** Composer, API client, Express (auth, validation, ownership), `chat.service`, `context.service`, `ai.service` / `gemini.provider`, Gemini, MongoDB (`messages`), UI.
- **Arrows:** as §17.1, including the failure path (Gemini error → error mapping → UI Retry) and "save user message first".

37.5 DFD Level 0 (context diagram)

- **One process:** 0 NeuroChat System.
- **External entities:** User; Google Identity Service; Gemini API.
- **Flows:** User → system: registration/login details, prompts, conversation commands; system → User: responses, history, errors; system ↔ Google: ID token / verified identity; system ↔ Gemini: prompt+context / generated text.

37.6 DFD Level 1

- **Processes:** 1.0 Authenticate User; 2.0 Manage Conversations; 3.0 Process Chat Message (build context + call AI); 4.0 Manage Profile & Settings.

- **Data stores:** D1 Users; D2 Conversations; D3 Messages.
- **Flows:** 1.0 ⇌ D1; 1.0 ⇌ Google; 2.0 ⇌ D2 and D3 (delete cascade); 3.0 reads D3 (history), writes D3, updates D2 (`updatedAt` , title), ⇌ Gemini; 4.0 ⇌ D1. Label each flow with the data it carries.

37.7 Use Case Diagram

- **Actors:** Visitor; Registered User (generalises Visitor); *secondary actors* Google Identity, Gemini API.
- **Use cases:** View Landing Page; Register; Login (with password); Login with Google (*associates Google*); Logout; Start Conversation; Send Message (*includes Build Context*; *includes Get AI Response* — *associates Gemini*); Retry Response; Copy Response; View History; Search Conversations; Rename Conversation; Delete Conversation (*includes Confirm Deletion*); Update Settings; View Onboarding; **MVP+** Use Voice Input/Output.
- **System boundary:** rectangle "NeuroChat".

37.8 ER Diagram

- **Entities:** User, Conversation, Message (with attributes from §22).
- **Relationships:** User 1—N Conversation ("owns"); Conversation 1—N Message ("contains"); User 1—N Message ("authored/owned"). Mark PKs, FKs, unique (`email` , `googleId`), and cardinality on each line.

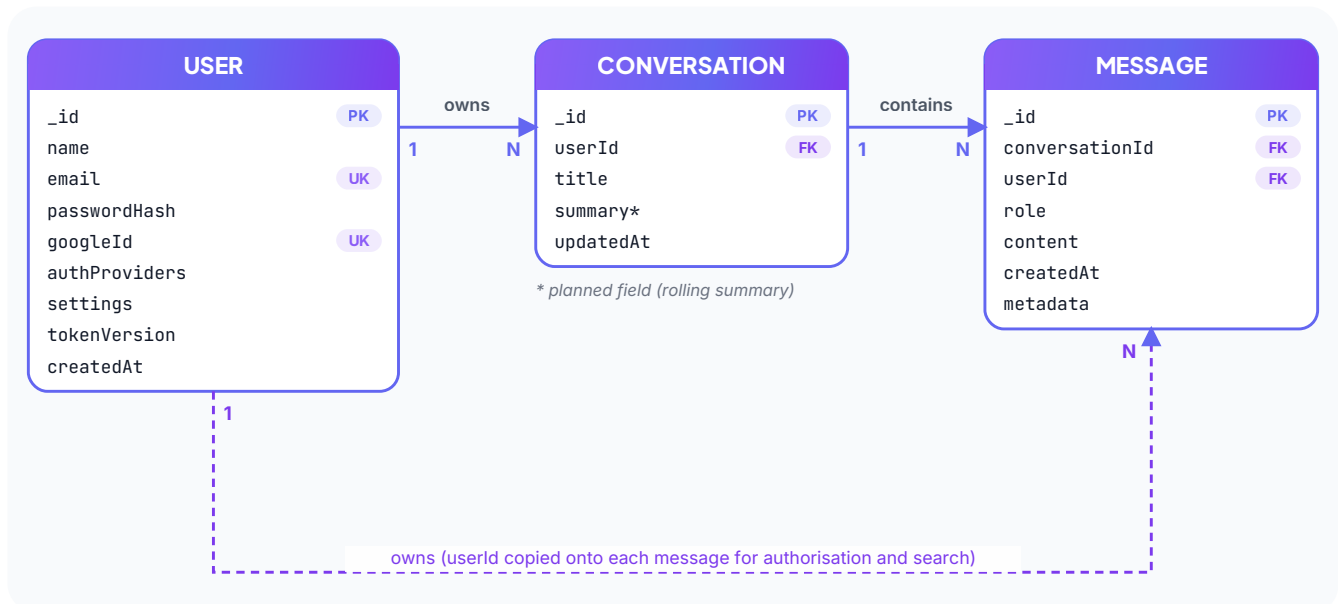


Figure 37.8 — Entity-relationship diagram

37.9 Sequence Diagram: Send Message

- **Lifelines:** User, React UI, Express (middleware), chat.service, context.service, gemini.provider, Gemini API, MongoDB.
- **Messages:** User types+sends → UI shows optimistic bubble → `POST .../messages` → middleware (auth, validate, ownership → MongoDB check) → save user message → fetch recent history → build context → generate → Gemini → response → validate → save assistant message → 201 → UI renders. Include an `alt` fragment: *Gemini error* → mapped error → UI shows Retry.

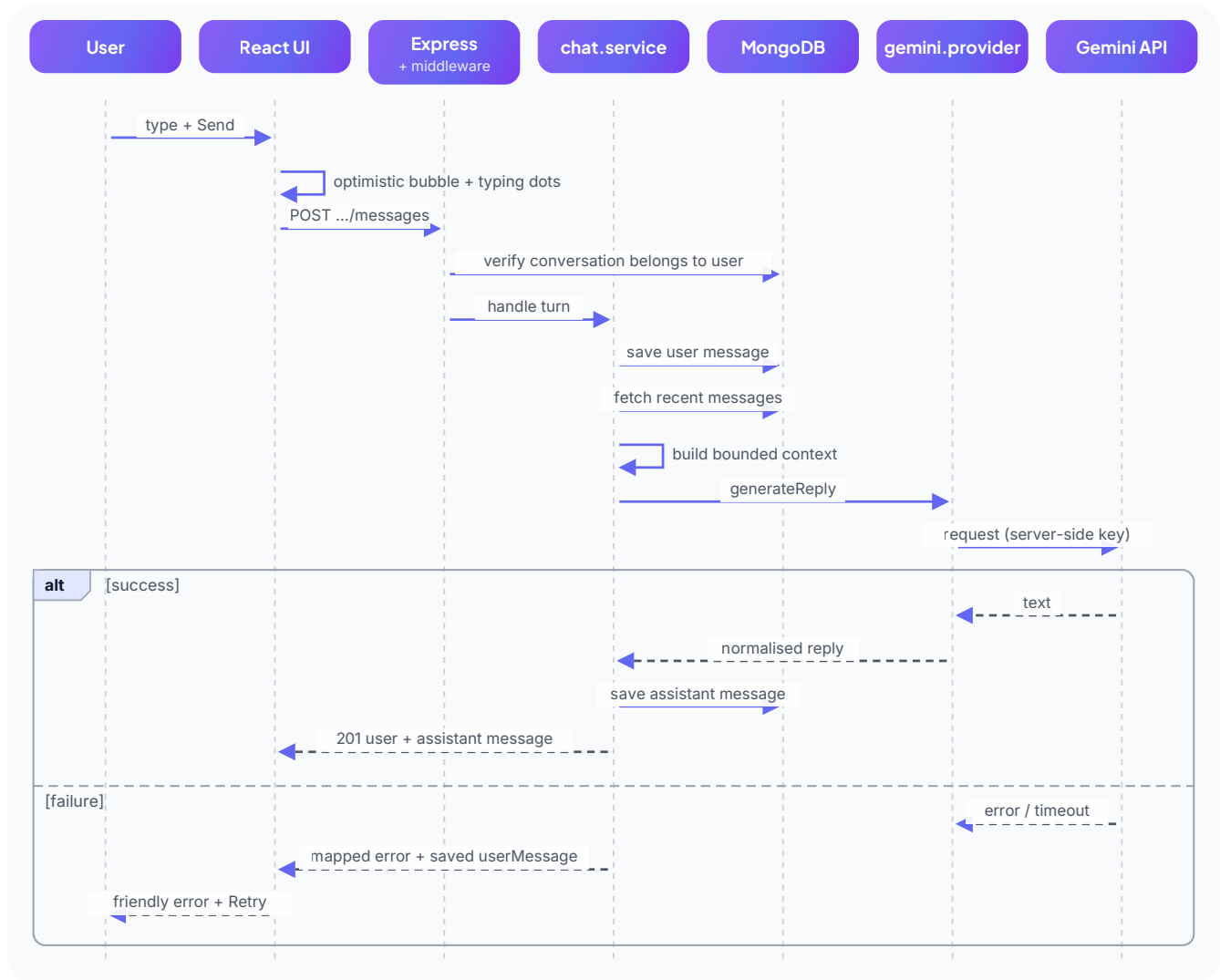


Figure 37.9 — Sequence diagram: send message

37.10 Sequence Diagram: Google Login

- **Lifelines:** User, React UI, Google (GIS), Express, google.service, Google public keys, MongoDB.
- **Messages:** as §37.3 steps, with **alt** fragments for new user / existing Google user / existing email account (linking rule).

37.11 Activity Diagram: Authentication

- **Swimlanes:** User · Frontend · Backend.
- **Flow:** Start → choose Register/Login/Google → (Register) enter details → client validation → [invalid → show errors, loop] → submit → server validate → [duplicate → error] → hash + create → issue cookie → (Login) submit → find user → compare → [fail → generic error] → cookie → (Google) as §37.3 → **merge** → redirect to `/chat` → End. Include rate-limit decision.

37.12 Activity Diagram: Chat

- **Swimlanes:** User · Frontend · Backend · Gemini.
- **Flow:** Start → open chat → new or existing conversation → type prompt → [empty? block] → send → backend validate/ownership → save user message → build context → call Gemini → [error? → error message → Retry loop] → validate response → save → display Markdown → [sensitive topic? show note] → continue or End.

37.13 Deployment Architecture

- **Elements (nodes):** User devices (desktop/mobile browser); Vercel (frontend build + `/api` rewrite); Render (Express service); MongoDB Atlas cluster; Google Cloud (OAuth client); Google AI (Gemini API).
- **Arrows/labels:** HTTPS everywhere; rewrite `/api/*` → Render; Render → Atlas (TLS, allow-listed); Render → Gemini; Browser → Google Identity; annotate where each environment variable lives (§43) and where **no secrets** exist (browser/Vercel build).

38 Academic Report Alignment

38.1 Core mapping (Semester 3 study → Semester 5 implementation)

Previous finding	NeuroChat implementation	Requirement IDs	Status
AI improves productivity (education, business)	Conversational assistant with saved multi-turn conversations	FR-011, FR-012, FR-013	MVP
Accuracy limitations	"AI-generated" labelling, disclaimer, uncertainty-aware system instruction, sensitive-topic nudge, retry, error handling	AI-003...AI-007, AI-011	MVP
Need for current information	Not claimed. Retrieval extension point and <code>sources</code> field; honest UI text	AI-009, AI-010	Planned
Regional language support	Unicode-safe storage, Hindi/English UI, reply-language preference	FR-028, NFR-007	MVP
Voice recommendation	Browser-based voice input and read-aloud module	FR-029, FR-030	MVP+
New-user guidance	First-run onboarding with example prompts and responsible-AI note	FR-027	MVP
User experience	Modern light-first responsive UI, loading/error states	UI-001...UI-010	MVP
Accessibility/usability	Accessibility specification and testing	UI-005...UI-008	MVP
Users need accounts and continuity	Email/password + Google authentication	FR-002...FR-009	MVP
Persistent use	Conversation history, search, delete	FR-013...FR-019	MVP
Responsible AI use	Trust section, honest copy, privacy page	AI-004, AI-010, NFR-010	MVP

38.2 Suggested narrative for the report's introduction

"In Semester 3 we studied ChatGPT and conversational AI through a survey of 80 participants and identified recurring limitations: inaccurate or general answers, lack of current information, limited regional-language support, absence of voice interaction, and insufficient guidance for new users. In Semester 5 we transformed that study into NeuroChat, a full-stack AI conversational assistant built with React, Node.js, Express and MongoDB, using the Google Gemini API through a secure backend. NeuroChat does not train its own model; its contribution is the design and implementation of the complete application and of the mechanisms that respond to the study's findings."

38.3 Suggested Semester 5 report chapters

1. Introduction and link to the Semester 3 study · 2. Requirement analysis (from §3, §8, §9) · 3. System design (architecture, DFD, ER, sequence diagrams) · 4. Database design · 5. Implementation (per module, with screenshots) · 6. Security · 7. Testing (the matrix with results) · 8. Deployment · 9. Results and evaluation

against objectives O1–O10 · 10. Limitations and future scope (§36) · 11. Conclusion · References · Appendices (API spec, test evidence).

38.4 Optional evaluation to strengthen the link (if time permits)

A small **follow-up usability check** with 5–10 people from the original survey audience (task completion, simple satisfaction question, whether the disclaimer was noticed). Report it *as what it is*: a small informal sample, not a statistically valid study.

39 Viva Demonstration Plan

Target duration: about 6 minutes 45 seconds (inside the 5–7 minute window). Rehearse with a timer at least three times.

39.1 Preparation (before the examiner arrives)

- ☐ Live URL **and** local fallback both running; `/api/health` pinged 5 minutes earlier.
- ☐ Two Google accounts ready (test-user listed if the OAuth app is in Testing mode); an unused email for registration.
- ☐ A pre-tested prompt set: "What is Java?" → "What are its advantages?" → a Hindi prompt. (Test that these do not hit safety filters or rate limits.)
- ☐ Compass open, connected with a **read-only** user; `neurochat` database visible; connection string hidden.
- ☐ Architecture diagram (§37.1) and requirement mapping (§38.1) on slides.
- ☐ Browser zoom 110–125%, notifications off, one clean window plus one private window.
- ☐ Recorded screen video as backup.

39.2 Script

#	Time	Step (from requirement)	What to do	What to say (one line)
1	0:00	Landing page	Scroll hero → features → trust section	"Built from our Semester 3 findings, including an honest limitations section."
2	0:30	Registration	Type a weak password (meter), then a valid one; register	"Validation, strength meter, hashed password on the server."
3	1:15	Google Sign-In	Log out; Continue with Google	"Google proves identity; my server verifies it; the frontend is not trusted."
4	1:45	Dashboard	Show onboarding card, dismiss	"Onboarding addresses the 'new users need guidance' finding."
5–6	2:05	New conversation, ask a question	"What is Java?"	"API key is only on the server."
7	2:20	AI response	Point at badge, Markdown, Copy	"Labelled AI-generated; rendered safely; never claims to be always right."
8	2:35	Follow-up (context)	"What are its advantages?"	"It understands 'its' because the recent messages go with each request."
9	3:05	History	Point to sidebar title	"Title generated from the first message, no extra AI call."
10–11	3:15	Refresh page	F5; open conversation	"Persisted in MongoDB."
12	3:25	Search	Type "jav"	"Debounced, scoped to my account."

#	Time	Step (from requirement)	What to do	What to say (one line)
13	3:40	Delete	Delete a throwaway chat; confirm	"Confirmation; messages removed too."
14	4:00	Logout	Click logout	"Cookie cleared."
15	4:10	Login again	Google or password; conversations present	"History follows the user." <i>(optional 10 s: log in as the other user and show an empty sidebar → isolation)</i>
16	4:30	MongoDB Compass	Switch to Compass; open <code>users</code>	"Password is a bcrypt hash; Google user has googleId."
17–18	4:50	User and conversation documents	Filter by email; then conversation	"One user owns many conversations."
19	5:15	Messages	Filter by conversationId; show Hindi message if used	"Roles alternate; Unicode is stored correctly; this is what Gemini receives as context."
20	5:45	Architecture	Show diagram §37.1	"React → Express → Gemini. Layers: routes, controllers, services, models. Nothing bypasses the backend."
—	6:30	Close	Requirement mapping slide	"Each Semester 3 finding became a requirement; here is what's done and what's planned."

If something fails live: stay calm, say what should have happened, switch to the local fallback or the recording. A visible, friendly error with Retry is itself a feature; don't hide it.

39.3 Likely examiner questions (short answers)

Question	Answer
Is this just a ChatGPT clone?	No. It's an application built around our own study's findings, with its own design, authentication, database and safety features. The AI model is Gemini; we did not train a model.
Why can't the frontend call Gemini directly?	The API key would be visible in the browser and could be stolen and abused. The backend holds the key.
How do you keep one user's chats private?	Every query filters by the logged-in user's ID taken from the verified token, never from the request; other users' IDs return 404. Tested.
Why a cookie instead of localStorage for the token?	JavaScript can't read an httpOnly cookie, so XSS can't steal the session.
Can someone fake Google login?	The backend verifies Google's signature, that the token was issued for my app, and that the email is verified. Frontend-supplied profile data is ignored.
How does the AI remember?	It doesn't remember by itself. For each message I send the latest messages of that conversation, within a size limit.
What happens in very long conversations?	Older messages fall out of the window and the user is told; summarisation is planned.
What if Gemini is down or the quota is exhausted?	Friendly error, Retry, user's message already saved, the rest of the app still works.
Is the AI always right?	No. The app says so and nudges users to verify important information.
How are passwords stored?	bcrypt hash with salt, cost 12; plaintext is never stored.
Why is <code>messages</code> a separate collection?	A conversation can grow large; embedding would hit size limits and hurt pagination and search.
Why does the message document also have <code>userId</code> ?	So ownership checks and search are a single indexed query; it costs 12 bytes per message.

Question	Answer
What's not implemented?	Web search/live data, citations, streaming, email verification, password reset, file upload. They're designed as extensions (§36).
What are the limitations?	AI accuracy depends on the model; free-tier quotas; context window forgets old messages; browser-dependent voice; no email verification yet.
How would it scale to many users?	Stateless API can run multiple instances; indexes and pagination keep queries fast; the practical bottleneck is Gemini quota, addressed with limits/queueing and a paid tier.
What did you learn / what would you change?	Prepare your own honest answer (e.g., adding streaming and email verification first).

40

Master Requirements Traceability Matrix

Links each requirement to its source (objective **O** from §6 or finding **F** from §3), the design section, the phase in which it's built, and the tests that prove it. Rows with no test in the matrix state how they're verified.

Req	Source	Design §	Phase	Verification
FR-001	O4, F1, F8	13.1	2	T-UI-01, T-UI-02
FR-002	O1	13.3, 15.3	3, 5	T-AUTH-01, 03, 04, 05
FR-003	O1	15.3	5	T-AUTH-02
FR-004	O1	15.4	5	T-AUTH-06, 07, 08
FR-005	O1	16	4, 5	T-AUTH-09, 10, 11, 12
FR-006	O1, O6	16.6	5	T-AUTH-13, 14
FR-007	O1	15.6	5	T-AUTH-15
FR-008	O1	15.2, 15.7	5	T-AUTH-16, 17
FR-009	O1, O6	15.7, 24.3	3, 5	T-AUTH-19, T-SEC-01
FR-010	O3	11.3, C1	7	T-CHAT-01
FR-011	O2	17.1, M2	7	T-CHAT-01
FR-012	O2, F7	19	7	T-CHAT-06
FR-013	O3	22	8	T-CHAT-11
FR-014	O3	C2	8	T-CONV-05
FR-015	O3	C3, M1	8	T-CHAT-11, T-SEC-05
FR-016	O3	11.4	7, 8	T-CONV-01
FR-017	O3	C4	9	T-CONV-02
FR-018	O3	C2, 30	9	T-CONV-03
FR-019	O3	C5, 22.7	9	T-CONV-04
FR-020	O2, O6	26.4	6	T-CHAT-05, T-SEC-07
FR-021	O2	11.3	6	T-CHAT-14
FR-022	F1	M3	7	T-CHAT-07, 08, 09
FR-023	O5	11.3	6	T-CHAT-02, 03, 04, 13
FR-024	F7	24.4	6	T-CHAT-17, T-UI-05
FR-025	O5	13.5	8	Visual check in T-UI-02

Req	Source	Design §	Phase	Verification
FR-026	O5	13.7, U1	9	T-UI-12
FR-027	F5	11.6	9	T-UI-11
FR-028	F3	20	11	T-UI-12, T-CHAT-16
FR-029	F4	21	11	T-VOICE-01
FR-030	F4	21	11	T-VOICE-02
FR-031	O5	14.11	11	T-UI-14
FR-032	O5	13.9	1, 11	T-UI-13
FR-033, FR-034	—	15.9	Planned	Not built; design only
AI-001	O2	17.1	7	T-SEC-08
AI-002	F7	19.3	7	T-CHAT-12 + unit tests
AI-003	F1	17.3	7	Manual review of sample answers
AI-004	F1, F8	17.3	6	Visual check in T-CHAT-01
AI-005	F8	17.3	7	T-CHAT-15
AI-006	F1	18.4	7	T-CHAT-07, 10
AI-007	O2	18.3	7	T-CHAT-10
AI-008	O7	18.6	7	Unit test with mock provider
AI-009	F2	17.4	7	Code review (interface exists, unused)
AI-010	F1, F2	13.1	2, 14	Copy-review checklist §45
AI-011	F1	18.4	7	T-CHAT-10
NFR-001	O5	30	11	Lighthouse (T-UI-08)
NFR-002	F7	30	7, 13	T-DEP-01 timing notes
NFR-003	O3	22.6, 30	8	T-CONV-05 + <code>explain()</code>
NFR-004	O7	24, 25	All	Code review
NFR-005	—	27	10	T-CHAT-07, T-UI-06
NFR-006	—	29	12	Manual smoke on 4 browsers
NFR-007	F3	20	11	T-CHAT-16, T-CONV-01
NFR-008	O9	44	14	Docs checklist
NFR-009	O8	25	5	Suites run offline with mocks
NFR-010	F8	26.9	2, 14	Privacy page review
SEC-001	O6	15, 26	5	T-AUTH-01, T-SEC-11
SEC-002	O6	15.2	5	T-SEC-13
SEC-003	O6	15.5	5	T-SEC-01, 02, 03, 04
SEC-004	O6	26.1	7–9	T-SEC-05
SEC-005	O6	23	5–9	T-AUTH-03, T-CHAT-04
SEC-006	O6	26.2	0	T-SEC-14
SEC-007	O6	26.3	10	T-SEC-09
SEC-008	O6	23.1	10	T-AUTH-18, T-SEC-15

Req	Source	Design §	Phase	Verification
SEC-009	O6	26.4	6	T-SEC-07
SEC-010	O1, O6	16.4	5	T-AUTH-11, 12
SEC-011	O6	26.5	10	Response-header inspection
SEC-012	O6	15.8	10	T-SEC-10, T-AUTH-07, 08
SEC-013	O6	26.1	5	T-SEC-06
SEC-014	O6	26.1	7, 12	T-SEC-08
UI-001	O5	14, 14.11	1	T-UI-01
UI-002	O5	14.5	1	Code review: grep for hex in components
UI-003	O5	14.8	11	Design checklist §14.10
UI-004	O5	29	11	T-UI-02
UI-005	O5, F6	28	11	T-UI-07, 08, 09
UI-006	F6	14.9	11	T-UI-10
UI-007	F6	27	6–11	T-UI-05, 06
UI-008	F6	29.2	11	T-UI-03
UI-009	O5	14.2	1	Asset review
UI-010	O5	14.11	11	T-UI-14

41

Acceptance Criteria

The project is **accepted for submission** when *all* of the following are true.

41.1 Functional

- ☐ A visitor can view the landing page, register, log in, log in with Google, and log out.
- ☐ A signed-in user can start a conversation, receive a Gemini answer, ask a follow-up that depends on context, and see both messages persist.
- ☐ After refresh and after logging out and in, the history is intact.
- ☐ The user can search, and delete conversations (with confirmation); auto-titles appear.
- ☐ Onboarding shows once and can be dismissed and reopened; UI works in English and Hindi.
- ☐ Every Must (M) requirement in §8–§9 has a passing test or documented verification in §40.

41.2 Security and AI honesty

- ☐ The Gemini key and JWT secret appear nowhere in the frontend bundle, network traffic, or Git history.
- ☐ User B cannot read or change User A's data (T-SEC-05 passes on all endpoints).
- ☐ Google login uses server-side token verification (T-AUTH-11/12 pass).
- ☐ No page or document claims guaranteed accuracy, live information, or a self-trained model.
- ☐ Every AI message is labelled and the disclaimer is visible.

41.3 Quality

- ☐ Default theme is light with subtle glow; design checklist §14.10 passed.
- ☐ Works at 360 / 768 / 1024 / 1440 px and on one real phone; demo completable by keyboard alone.

- ☐ axe/Lighthouse show no critical accessibility failures.
- ☐ Every data view has loading, empty, and error states.
- ☐ No temporary stub or dead link remains (e.g., "Forgot password" without a feature).

41.4 Delivery

- ☐ Live HTTPS deployment passes T-DEP-01; local fallback and recording exist.
- ☐ `docs/` set complete and consistent with the code; README lets a stranger run the project.
- ☐ Demo runs in ≤ 7 minutes; question bank rehearsed.

42

Risks and Mitigations

#	Risk	Likelihood	Impact	Mitigation	Contingency
R1	Gemini free-tier quota/rate limits hit during development or viva	Med	High	Per-user limits, concurrency guard, short prompts, cache nothing sensitive; test prompts sparingly	Keep a second key/project ready (per Google's terms); local recording
R2	Gemini model names/SDK change or deprecate	Med	Med	<code>GEMINI_MODEL</code> env variable; provider abstraction; verify docs at implementation	Swap model in env; one-file change
R3	Google OAuth misconfiguration (<code>origin_mismatch</code> , Testing-mode test-user restriction)	High	High	Follow §16.2 exactly; add all test users or publish; test the deployed origin early (Phase 4/13)	Password login still works; recorded Google demo
R4	Free backend host sleeps (slow first request)	High	Med	Ping <code>/api/health</code> before viva; optional uptime pinger	Run locally
R5	No/poor internet at viva venue	Med	High	Local fallback with local or Atlas DB; phone hotspot	Screen recording
R6	Cross-domain cookie blocked by browser	Med	High	Same-origin proxy (D3) or single-origin deploy	Switch to Express-serves-React deployment
R7	Secret accidentally committed	Med	High	<code>.gitignore</code> first, <code>.env.example</code> , gitleaks, no keys in screenshots	Rotate keys immediately; rewrite history if needed
R8	Scope creep (voice, dark mode, extras) delays core	High	High	MVP/MVP+ split; cut order §34.1; feature freeze before Phase 12	Drop <code>MVP+</code> items
R9	AI gives a wrong/odd answer or is blocked during demo	Med	Med	Pre-test demo prompts; treat as a chance to show disclaimer and Retry	Rephrase, retry, or use a prepared conversation
R10	Examiner sees it as "a ChatGPT clone"	Med	Med	§38 mapping, own branding, discussion of findings → features	Lead the viva with the mapping slide
R11	Privacy concern: user text goes to a third-party AI	Med	Med	Truthful Privacy page; onboarding warning; no sensitive data guidance	State it proactively in viva
R12	Time underestimated for backend/auth/testing	High	High	Phase plan with cut order; get auth + chat working end to end early (Phases 5–7)	Reduce polish, not correctness or security
R13	Atlas access blocked (network/DNS, IP allow-list) on venue network	Low–Med	Med	Test Compass connection from another network; hotspot	Local MongoDB copy with seeded data
R14	Hindi text rendering/search issues	Low	Med	Fonts, <code>default_language: "none"</code> , Unicode tests	Restrict claims to what tests show

#	Risk	Likelihood	Impact	Mitigation	Contingency
R15	Dependency vulnerabilities or breaking updates	Med	Low	Lock files; <code>npm audit</code> ; avoid upgrading in final week	Pin versions

43 Environment Configuration

Never put real values in this document or in Git. Generate a strong secret with: `node -e`

`"console.log(require('crypto').randomBytes(48).toString('hex'))"`.

43.1 Backend (backend/ .env.example)

```
# --- Server ---
NODE_ENV=development
PORT=5000
CLIENT_URL=http://localhost:5173      # exact frontend origin, no trailing slash
SERVER_URL=http://localhost:5000      # set 1 in production behind a proxy
TRUST_PROXY=0

# --- Database ---
MONGODB_URI=                          # Atlas or local connection string (contains credentials: keep secret)

# --- Auth ---
JWT_SECRET=                           # >= 32 random bytes
JWT_EXPIRES_SHORT=24h                 # session without "remember me"
JWT_EXPIRES_LONG=30d                 # with "remember me"

# --- Google ---
GOOGLE_CLIENT_ID=                     # public client ID (same value as frontend)
GOOGLE_CLIENT_SECRET=                 # OPTIONAL and UNUSED in ID-token flow; only for the code-flow
alternative

# --- Gemini ---
GEMINI_API_KEY=                       # secret; backend only
GEMINI_MODEL=                         # a current Flash-class model name from Google AI docs
AI_TIMEOUT_MS=30000
```

Variable	Required	Secret?	Purpose
<code>NODE_ENV</code>	Yes	No	Controls error verbosity and cookie Secure
<code>PORT</code>	Local (host may inject)	No	API port
<code>CLIENT_URL</code>	Yes	No	CORS allow-list and Origin check
<code>SERVER_URL</code>	Optional	No	Absolute URL of API (logging, future callbacks)
<code>TRUST_PROXY</code>	Production	No	Correct client IP behind proxy
<code>MONGODB_URI</code>	Yes	Yes	Database connection
<code>JWT_SECRET</code>	Yes	Yes	Signs session tokens
<code>JWT_EXPIRES_*</code>	Optional	No	Token lifetimes
<code>GOOGLE_CLIENT_ID</code>	Yes	No (public)	Verify token audience
<code>GOOGLE_CLIENT_SECRET</code>	Optional/unused	Yes	Only if switching to code flow
<code>GEMINI_API_KEY</code>	Yes	Yes	AI access
<code>GEMINI_MODEL</code>	Yes	No	Model selection
<code>AI_TIMEOUT_MS</code>	Optional	No	Gemini timeout

43.2 Frontend (frontend/.env.example), public values only

```
VITE_API_BASE_URL=/api           # same-origin via dev proxy / production rewrite
VITE_GOOGLE_CLIENT_ID=          # public Google client ID
```

Rule: anything prefixed `VITE_` is embedded in the browser bundle and visible to everyone. **Never** put `GEMINI_API_KEY`, `JWT_SECRET`, `MONGODB_URI` or `GOOGLE_CLIENT_SECRET` in a `VITE_` variable or in any frontend file.

43.3 Where each lives

Location	Holds
Developer machine <code>.env</code> (git-ignored)	All backend secrets for local development
Render (backend host) dashboard	All backend variables in production
Vercel (frontend host) dashboard	Only <code>VITE_*</code> public values
Google Cloud Console	OAuth client, origins, consent screen
Atlas	DB user credentials, network access
Git repository	<code>.env.example</code> files only

44 Documentation Strategy

44.1 The docs/ set

```
docs/
├─ MASTER_SPECIFICATION.md  This document: source of truth, frozen after approval; amend deliberately
├─ REQUIREMENTS.md          $8-$9 tables + live status (Not started / Done / Verified) per requirement
├─ ARCHITECTURE.md          Layers, module responsibilities, diagrams (§37.1, 37.4, 37.13), key decisions D1-D12
├─ DATABASE.md              Collections, fields, indexes, ER diagram, sample documents, Compass guide
├─ API.md                   $23 kept in sync with routes; example requests/responses (copy from real runs)
├─ AUTHENTICATION.md        Cookie/JWT design, registration/login flows, CSRF, tokenVersion
├─ GOOGLE_AUTH.md           Console setup steps with your real screenshots (blur IDs), flow, linking rules
├─ AI_INTEGRATION.md        Gemini service, system instruction text, context algorithm, error mapping, retrieval
roadmap
├─ UI_UX.md                 Design tokens, component catalogue, page specs, accessibility and responsive notes
├─ TESTING.md               Test matrix with real Actual Result/Status, evidence screenshots, coverage notes
├─ DEPLOYMENT.md            Exact steps, env var locations, post-deploy checks, fallback plan
└─ VIVA.md                  Demo script, timings, question bank, fallback procedure
```

44.2 Rules

- Specification first, then code.** If implementation forces a change, update `MASTER_SPECIFICATION.md` (or the specific doc) in the same commit and note it in a `CHANGELOG` section at the end of this document.
- One fact, one home.** Don't copy tables between documents; link to them.
- Statuses are honest.** Only mark **Done** when it runs; only mark **Verified** when a test in `TESTING.md` passed.
- Evidence:** keep screenshots (network tab without keys, Compass, responsive views) in `docs/evidence/` (never showing secrets).
- README.md (root):** project summary and screenshot, feature list with `MVP` / `Planned` honesty, prerequisites, local setup (clone, `.env` from `.env.example`, `npm install`, run both servers), test commands, deployment link, credits, licence.
- Keep documents in Markdown so they render on GitHub and can be exported to PDF/DOCX for the college.

44.3 Changelog (maintain here)

Date	Change	Reason
2026-09-20	v1.0 baseline specification	Initial

45

Final Implementation Checklist

Use this as the last gate before submission. Every box must be checked or have a written reason.

Foundations

- ☐ Repo has `.gitignore` covering `.env*`, `node_modules`, build output; **no secrets in history** (gitleaks clean)
- ☐ `.env.example` for backend and frontend with placeholders only
- ☐ All temporary stubs removed; no `console.log` of tokens, keys, or message content

Authentication and security

- ☐ Register / login / Google / logout / session restore work
- ☐ Passwords bcrypt-hashed; Google ID token verified server-side (audience + `email_verified`)
- ☐ Linking rules of §16.6 implemented and tested
- ☐ httpOnly + Secure + SameSite cookie; Origin check; rate limits; helmet; CORS allow-list
- ☐ Ownership checks on **every** conversation/message endpoint (T-SEC-05 all pass)
- ☐ Input validation on every endpoint; NoSQL-injection test passes
- ☐ Markdown sanitised; images disabled; links hardened
- ☐ `npm audit` reviewed

AI and conversations

- ☐ Gemini called only from backend; key absent from bundle (`grep dist/`)
- ☐ Context window with budget implemented and unit-tested; truncation notice shown
- ☐ Friendly errors for 429 / timeout / 503 / empty / blocked, each with Retry
- ☐ AI-generated badge, disclaimer, sensitive-topic nudge present
- ☐ History persists; search, delete-with-confirm, auto-title work; foreign IDs return 404

Honesty review (AI-010)

- ☐ No claim of live/real-time data, guaranteed accuracy, or an own AI model anywhere (landing, onboarding, README, report, slides)
- ☐ Every non-built feature labelled **Planned** / **Future**
- ☐ Privacy page accurately describes Gemini and Google data flows and current deletion capabilities

UI / UX / accessibility

- ☐ Light-first default verified on a dark-mode OS; glow subtle; design checklist §14.10 passed
- ☐ Landing page complete (hero, features, how it works, why NeuroChat, use cases, trust, footer)
- ☐ Onboarding works; English + Hindi UI; Unicode round-trip verified
- ☐ Mobile drawer, composer, and forms verified on a real phone
- ☐ Keyboard-only run of the demo; axe/Lighthouse recorded; reduced motion respected
- ☐ Loading / empty / error state present on every data view; no raw stack traces anywhere

Testing and deployment

- ☐ Test matrix executed; *Actual Result* and *Status* filled; failures fixed or documented
- ☐ Live deployment passes T-DEP-01; Google origin configured for production
- ☐ Local fallback runs; screen recording made; `/api/health` pinged before viva

Documentation and viva

- ☐ All 12 files in `docs/` present and consistent with code; README verified by a fresh clone
- ☐ Diagrams §37 drawn and captioned; requirement mapping §38 in report
- ☐ Demo rehearsed ≥ 3 times within 7 minutes; question bank (§39.3) rehearsed
- ☐ Appendix B items filled in

A

Internal Consistency Review

Performed before finalising, as requested. "Result" reflects this document's own consistency, not that of code that doesn't yet exist.

Check	Result	Evidence / how resolved
Do requirements match architecture?	✓	Each FR/AI/SEC/UI item maps to a section and phase in §40; no requirement lacks a design home
Does the database design support all required features?	✓	Users (auth, settings, onboarding, tokenVersion), conversations (title, ordering), messages (roles, metadata, <code>userId</code> for isolation/search); indexes cover list/history/login/Google lookups
Does the API design support the frontend?	✓	Every page action in §13 has an endpoint in §23; cursor pagination, retry, and settings included. <i>Adjusted</i> : lazy conversation creation is a two-call sequence (C1 then M2) rather than one combined endpoint, to keep endpoints single-purpose
Does authentication support Google Sign-In?	✓	<code>googleId</code> , <code>authProviders</code> , <code>emailVerified</code> , linking rules, one session mechanism for both methods
Does the AI architecture support conversation context?	✓	§19 algorithm, <code>contextTruncated</code> flag, unit-testable context service
Are security requirements addressed?	✓	SEC-001...014 each mapped to controls (§26.1) and tests (§31.2)
Are previous report drawbacks addressed?	✓ / partial	F1, F3, F5–F8 in MVP; F4 in <code>MVP+</code> ; F2 is deliberately not solved : an honest architecture only, per instruction
Are MVP and future features clearly separated?	✓	Status tags throughout; §35–§36
Is the light glowing UI reflected throughout?	✓	UI-001...003, §14, §13 page specs, Phase 1 and 11 checks; dark mode is optional and secondary
Can it realistically be built and demonstrated by a BCA Semester 5 student?	✓ with discipline	MVP is a defined subset; cut order in §34.1; single-origin deployment fallback

Tensions found and how they were resolved

1. `GOOGLE_CLIENT_SECRET` in your env list vs the chosen Google flow. ID-token verification doesn't use a client secret. It's kept as optional/unused so the alternative flow remains possible.
2. "Redirect URI/callback configuration" requirement. The JavaScript-callback flow needs *authorised origins* but no redirect URI. Both cases are documented (§16.2).

3. **Email/password accounts aren't email-verified in the MVP**, which creates the account-pre-hijacking risk. Resolved in §16.6 by trusting Google-verified email and discarding unverified local passwords.
4. **"Do NOT duplicate large data" vs `userId` on messages**. Only a 12-byte ObjectId is duplicated, in exchange for simple, safe authorisation queries (D8).
5. **Phase order in your prompt builds UI before the backend**. Kept, but with clearly-marked temporary stubs that must be deleted (§34.1).
6. **Voice**. Marked **MVP+**, not required, because browser support varies and it must not endanger required features.
7. **Cross-domain cookies vs separate frontend/backend hosting**. Resolved by same-origin proxy with two fallbacks (§32.3).

Limits of this specification

- It was written without seeing your Semester 3 report, so **no survey statistics are included**.
- Library names, model names, hosting free-tier terms and Google Console labels change often; each place where this matters says **verify at implementation time**.
- Time estimates are planning aids only.

B

Items You Must Fill In From Your Own Work

Item	Where it's used
Semester 3 report title, guide name, college and university name, academic year	Title page, README, footer "About"
Survey results (at least the 3–4 headline percentages), participant profile	§3, landing "Why NeuroChat", report Chapter 1
Exact wording of the eight findings as written in your report	§3 table (align terminology)
Production URLs (frontend, backend)	§32, README, Google origins
Chosen <code>GEMINI_MODEL</code> value (from current Google AI documentation)	§18.1, <code>.env</code>
Contact email for footer and Privacy page	§13.1
Your own "what I learned / what I'd change" answer	§39.3
Real test results and screenshots	<code>TESTING.md</code> , §31.2

End of MASTER PROJECT SPECIFICATION, NeuroChat — Your Intelligent Conversation Partner (v1.0).